



An investigation into the automated simplification of UK legislative texts

Archie Harrodine

2127935

Supervised by James Archbold

University of Warwick

April 2024

Abstract

With the complexity of laws and legislation, it is difficult for the average British citizen to remain up to date and aware of developments in the country's governance. Advancements in large language models and text generation now enable us to combat this, simplifying the laws passed into something more accessible to read. This report was an investigation into developing an algorithm that would interpret and simplify UK law, legislation, and bills, to improve the transparency between the UK populace and the Government. We study the potential viability of an application that would make this possible, and the ethical requirements that would need to be met to justify its release.

Keywords— Text Simplification, Large Language Models, Readability, Legislation, Text Generation

Glossary

Legislation: Documents describing the law.

Bill: A draft of a proposed law presented to parliament for discussion.

Act: A passed law which is either enacted or to be enacted.

Text simplification: The reduction of lexical or syntactic complexity of text.

Text summarisation: The reduction in size of text, condensing the main ideas of a text.

Acknowledgements

I would like to thank my supervisor James Archbold for all the support and guidance he provided throughout the project. His feedback and advice were invaluable. Without him the project wouldn't have made it as far as it did.

I would also like to thank my friends and family for supporting me through every step of this project, especially those who participated in the testing. Particular thanks to Owen Green for helping me solve a significant API problem.

Special thanks to Asher Harvey-Smith, Ivan Kudryavtsev, Morgan Kippen and Icen Zeyada, for being incredible house mates and being there for me during this project.

Finally, I would like to thank my parents for the constant moral support and help that they provided, and I wouldn't have accomplished this without them.

Contents

1	Background	
1.1	Introduction	
1.2	Motivation	
1.3	Objectives	
1.3.1	Module 1: Text simplification	
1.3.2	Module 2: Political sentiment analysis	
1.3.3	Module 3: Demographic and topic identification	
1.3.4	Module 4: Live updating system	
1.3.5	Module 5: Data storage	
1.3.6	Module 6: User interface	
1.3.7	Module 7: Ethical discussion	
2	Research	
2.1	UK legislation	
2.1.1	Types of legislation	
2.1.2	Introducing bills	
2.1.3	Legislation structure	
2.2	Evaluation metrics	
2.2.1	Readability	
2.2.2	Summarisation	
2.2.3	Orthographic and syntactic	
2.2.4	Bias	
2.2.5	Information theory	
2.3	Text simplification	
2.3.1	Lexical simplification	
2.3.2	Syntactic simplification	
2.3.3	Extractive summarisation	
2.3.4	Abstractive summarisation	
2.4	Large language models	
2.4.1	Emergent abilities in LLMs	
2.4.2	LLM selection	
2.4.3	LLM prompting	
2.4.4	Chunking problem	
2.5	Existing solutions	
3	Implementation	
3.1	Design	
3.2	Bill and Legislation APIs	
3.2.1	Legislation API	
3.2.2	Bills API	
3.3	Parsing	
3.4	Chunking	
3.5	Generation	
3.5.1	Model loading	
3.5.2	Context window	
3.5.3	Summariser	
3.5.4	Prompt engineering	
3.5.5	Hyperparameter tuning	
3.6	Metric evaluation	
3.6.1	Readability	
3.6.2	Summarisation	
3.6.3	Orthographic and syntactic	
3.6.4	Sentiment	
3.6.5	Information theory	
4	Results	

4.1	Key metrics	
4.2	Recursive and TextRank	
4.3	Impact of length	
4.4	Qualitative analysis	
5	Project management	
6	Ethical discussion	
7	Evaluation	
8	Conclusions	
9	Appendices	
9.1	Score contexts	
9.2	Prompts	
9.3	Configs	

1 Background

1.1 Introduction

Law is complicated. Legal documents are long, highly specialised pieces of text with dozens of references to other complex texts. They are full of legal jargon, with levels of detail and scrutiny that makes it difficult to decipher the text’s meaning. Very few people understand these texts, and fewer are inspired to try. Yet everybody on the planet is impacted by the laws passed every day. They impact people’s freedoms and rights, and yet they are buried in legal documents that so few can read. We see this as a systemic issue that needs to change, and with the advancements found in natural language processing (NLP), we have investigated methods to make legal documents more accessible to the wider public. With our focus on UK legislative texts, we explored research in creating an application which summarises and explains these texts, while retaining as much of the original meaning as possible.

1.2 Motivation

The need for a method to better understand the law is well documented, with many studies pointing toward “substantial knowledge deficit” regarding the public’s understanding of the law[134]. With 34% of those experiencing legal problems claiming to not understand their legal position at all[134] and Legal Services Board Chair Dr Helen Phillips saying that “Many fail to identify the issues they face as being legal in nature”[9], it is clear that a thorough understanding of the law is not common amongst the public. This issue is exacerbated when we consider we live in a ‘law-thick’ world[49], where our everyday actions take place within a complex legal framework[134]. Hodge Jones & Allen found an even stronger connection, with 73% of people having little understanding of how the legal process works[1]. This lack of legal knowledge can be attributed to various factors, especially that the majority of people do not need to recall this information on a regular basis. However, this isn’t just a case of not understanding the minutiae of law, as these studies demonstrate that many find it difficult to even identify what is or isn’t a legal issue. This lack of education is exemplified with The Law Society heavily promoting public legal education[166]. Although the law is far more accessible in recent years with websites like <https://www.legislation.gov.uk/>[87], legislation remains difficult and complex, described as either “drafted in another era” or being “repeatedly amended over”[6]. Even if we look at modern legislation which has scarcely been amended, the text statistics of an average piece of legislation depict the text to be very difficult to read(1).

Metric	Average (2 d.p.)
Number of words	9288.32
Number of sentences	265.74
Average of words in a sentence	32.68
Flesch reading ease	38.19
Flesch-Kincaid grade	16.05
SMOG	16.18
Gunning-Fog index	19.62
ARI	19.74

Table 1: List of key metrics of 670 UK legal documents from between 1995 and 2023.

The average word count suggests the average UK adult will spend upwards of an hour reading the text[186]. Furthermore, the average number of words in a sentence imply that each sentence is inherently difficult to read, as when compared to the advised sentence length of 15-20 words[158, 84], these documents possess an average of 32 words. Even the UK government sets a sentence length limit of 25 words for GOV.UK content[181]. Lastly, the average reading grade of these texts, gathered from several readability measures[78, 164, 46, 52], suggest a need for 18 years of education, corresponding to beyond university graduate level(14).

Considering this information in the context of the UK, the average reading age is only 9 years of education[175]. Additionally, 62.7% of the UK’s population aren’t university graduates[47], hence we see

how much of the population is locked out of being able to read and understand these legal documents. We aren't even accounting for the 8.9% that don't have English as their first language[107], or the 10% of the populace affected by dyslexia[182]. Not only would a significant portion of the population struggle to read them, but there is the added difficulty of the sheer volume of texts to read through. There is no easy way to identify what laws - or bills to be passed - may impact you and your life. Thus, we need to find a better way to categorise and summarise UK legislative texts, to make the information hidden under layers of jargon, more accessible to the public.

Until recently, qualitative natural language tasks were difficult, with limited methods to interpret and generate texts. With the developments in text generation and large language models (LLMs), we now have techniques to harness natural language in our programs more effectively[185]. Combining LLMs and other text simplification techniques, we are able to create better legal summaries for our users.

1.3 Objectives

Our ultimate aim was to create an application that would improve the public's relationship with UK law, facilitating transparency between the UK populace and the government. We defined a multitude of methods we could accomplish this, detailing features that would best serve the public. The largest issue we identified was the readability of UK legislation, of both acts and bills. Therefore, our project was oriented around the simplification and summarisation of UK legislation, making them easier to read, by reducing the average reading age of the documents as low as possible while retaining the key aspects of the text. With our constrained time frame, we decided for many aspects of our objectives to be modular, so that as the project developed we could expand our scope accordingly. We assigned levels of priorities to our objectives using the MoSCoW system[22].

(M) Must have: essential requirements for the project.

(S) Should have: important requirements that are not critical.

(C) Could have: Advantageous requirements that are not necessary but are desirable.

(E) Extensions: Requirements that are outside the considered scope of the project.

1.3.1 Module 1: Text simplification

A module to simplify text so that it is easier to read. This is the primary focus of the project, and studies the simplification of legal texts. This module covers both bills and acts. While they tread similar formatting, the various changes and amendments a bill goes through differentiates how we can simplify it. It is equally important to be able to simplify both types of legal texts. The public needs to understand the laws that are currently impacting them, as well as the laws that are currently being passed, and may affect them in the future. Additionally, bills are often amendments of existing laws, meaning to truly understand one, we must have the background knowledge of the other.

Requirements:

1. We can extract a raw text form of all bills and legislation. (M)
2. We can pass the raw text through a model, and extract resultant text with a significant improvement to selected readability metrics. We aim to reduce the average readability of documents to the average readability of the UK, 9 years of education. (M)
3. Resultant text is accurate to the information within the original text, maximising key information retained, while minimising misinformation and hallucinations. (M)
4. Resultant text can describe the original text with multiple levels of detail, providing more information to users who may want it. (S)
5. Resultant text can describe how the legislation impacts a specific demographic. (C)

1.3.2 Module 2: Political sentiment analysis

A module to identify the tone of the text, and if responses have political leanings. This module is aimed to address some of the inherent ethical risks in our model being politically biased. The resultant text created could write critically of laws and the parties that passed them, which means the application could be used as a political weapon. This has the prerequisite of text simplification being completed.

Requirements:

1. A function to quantify the sentiment of a text, how positive, negative or neutral it is. (C)
2. A method to identify suitable resultant texts, which are a similar sentiment to the raw text, and are labelled neutral. (C)
3. A function to analyse the political leanings of the text using the left-right political spectrum. (C)
4. A function to analyse the political leanings of the text using the Eysenck political spectrum[34]. (E)

1.3.3 Module 3: Demographic and topic identification

A module to identify which topics are covered, and what demographics are impacted, by a given piece of legislation. This module focuses on the issue of finding the most relevant acts and laws to the user. Topic identification would be used to group texts so they are easier to find, whereas demographic identification would be used to alert relevant demographics of bills which may impact them. Topic identification is easier, using contents found entirely within the text, whereas demographic identification requires possible inferences to deduce.

Requirements:

1. A function to identify a list of key topics which a given text covers, in the form of synonymous words or short phrases. Accuracy would be measured by sample human reading. (C)
2. We need to research a list of all relevant demographics or groups which may be impacted by an introduced bill or law. (E)
3. A function to identify a list of key demographics which are impacted by the legislation. Demographics will be taken from the previously researched list. (E)

1.3.4 Module 4: Live updating system

A module to keep track of currently ongoing bills, and update relevant users to any changes that occur in parliament. This would focus entirely on the bills, providing a variety of important details such as their current stage, their sponsor, and what the bill would actually do. The majority of this information can be accessed through the UK Bills API[120], but will need to be processed and stored. Additionally some method of regularly checking the website API for updates may be needed.

Requirements:

1. A function to retrieve all accessible bill data, including short title, long title, stage, sponsor, relevant dates, and raw text. (S)
2. A function to retrieve all accessible act data, usually the complete document. (S)
3. A method to format all retrieved data, and easily feed into other modules. (S)
4. A function to simplify a given bill, with respect to the previous stage, expressing how it has changed. (C)
5. A method to send notifications to all impacted demographics (prerequisite of demographic identification). (C)
6. An assurance the application is kept up to date and accurate with current information. (S)

1.3.5 Module 5: Data storage

A module on storing all relevant information our application needs. If we want to include features of notifying affected demographics of specific bill updates, then a user database will be required. The user data would be entered through self-identification, designating details such as age, gender, occupation, tax bracket, etc. This would then need to be securely stored, to not violate GDPR. We would use SQL to store this information, as to make demographic queries far easier, whereas results from other modules would be stored in appropriate text, JSON or pickle files.

Requirements:

1. Ensuring security for the storage of given user data. Assure there will be no violations of GDPR. (M)
2. Storing self-identified user data. This includes personal details such as: age, occupation, gender etcetera. (S)
3. A system to store relevant information about a given bill or act. This would need to optimise space, not storing the original input text to reduce storage requirements. (S)
4. Modular design of data such that new details and attributes can be easily added. (S)

1.3.6 Module 6: User interface

A module to act as the front-end application, allowing users to interact with the other of the modules. As we aim to make legislation more accessible to users, a large part of that is making the documents easy to read, and making it engaging to users. We need to ensure that the interface is both interactive, simple and has several accessibility options. We aim to create the application as a phone app, with mobile phones being the most used device to go online[112]. The UI aspect is not the driving force of the project, as this isn't as important to design, and we only aim to build the framework for an ideal application, thus this module was given the least priority.

Requirements:

1. An application interface is created which allows users to read results from other modules, including simplifications and relevant text statistics. (M)
2. A method to access the original bill from within the application. (M)
3. The application is easy to use, engaging and interactive for the users. (S)
4. Multiple accessibility options built into the application. (C)
5. A functional and working mobile application. (C)

1.3.7 Module 7: Ethical discussion

A module to reach a conclusion on whether it is ethical to release this app, and the place of AI technologies in law and politics. The nature of this project means that the results of an algorithm could impact or influence a person's life significantly, whether in their political leanings or their legal standing. Throughout we make efforts to research how to design our algorithms and models such that we minimise inaccuracy and bias. However, we have to question if minimising these elements will be enough to justify the potential risk that is carried by this area of programming. This concerns the results of algorithms used, and how the app may be used. The project is being used as a medium to further investigate the future of this technology, and conclude if it is ever moral or ethical to use technology in this way.

Requirements:

1. Research the use of technology, AI, and automation in the realm of law and politics. (S)
2. Evaluate if it is ethical to use these technologies in law and politics. (S)
3. Evaluate if it can ever be safe to use these technologies in law and politics. (S)

4. Evaluate if AI technologies can ever truly be unbiased in politics. (S)
5. Use evidence of existing case studies, and use findings from the results of this project to justify our conclusion. (S)

2 Research

The first phase of our project was centred on thoroughly researching the core fields of law, text simplification and LLMs. We strived to understand these topics, to best decide upon the methods we could use to find solutions to our requirements.

2.1 UK legislation

To make any significant progress on simplifying and summarising UK legislation, we needed an understanding of what texts that entailed, how we could access them, and what were the most important aspects of the text to focus on. Our first steps saw us researching the fundamental aspects of law creation and structure, with the majority of information gathered through the UK government website[131, 115, 44].

2.1.1 Types of legislation

A bill is a proposal for a new law, or a proposal to change an existing law; whereas acts are laws that have been passed and are now in effect.

2.1.1.1 Bills

Bills are introduced in the House of Commons or House of Lords for examination, discussion and amendment. When both houses have agreed on the content of a bill, it is presented to the reigning monarch for ascension, then it becomes a law. Different types of bills can be introduced by: the government, individual MPs, Lords, private individuals, or organisations. There are several types of bills, each which interact with the passing of the law differently.

Public bills: Changes the law as it applies to the general population. They are the most common type of bill and are proposed by government ministers. Members of both houses closely examine the bill. While they can be introduced in either house, bills that are financial or involve the public's money, are always introduced in the House of Commons. A public bill can be opposed, by writing to a MP or Lord; writing to the government department responsible for the bill, lobbying parliament, or submitting evidence to the relevant public bill committee.

Private member's bills: Public bills that are introduced by MPs or Lords who are not ministers. As opposed to public bills, complete legal texts aren't required for each stage; only a short and long title. Usually this bill has reduced priority, and is introduced less formally in a variety of ways. Private member's bills raised by lords have less time devoted to them.

Private bills: Prompted by organisations like authorities or private companies, often to change the law as it applies to specific individuals or organisations. Groups who may be impacted by this bill can petition parliament against the proposed bill and present their objections to the committees of MPs and the Lords. Private bills however, often are only publicised through newspaper adverts, and writing to all interested parties.

Hybrid bills: Changes proposed by a hybrid bill impact both the general public, and specific individuals or groups. Usually introduced by the government, they often take a long time to pass.

2.1.1.2 Acts

Acts are bills which have been passed by both the House of Commons and the House of Lords. They describe the specifics of the law, what they do, and how it should be enforced. Acts can be from any type of bill, with its format remaining mostly consistent between texts.

Primary legislation: These are acts of parliament which have been passed, and are either being enforced, or are being implemented.

Secondary legislation: This generally refers to statutory instruments, which can be orders, regulations, rules, codes etc. They are acts which are introduced alongside ascending bills as additional laws to assist the original act. These are typically introduced by government ministers, and give far more specific details with respect to aspects of the original act.

2.1.2 Introducing bills

While each type of bill has a slightly different process it must go through to be passed, the majority follow a fairly similar linear structure.

- **First Reading:** A short title is announced, and is printed as a paper.
- **Second Reading:** A debate is opened to the MPs for the main principles of the bill. At the end of the debate, a vote is held to see if it progresses to the next stage.
- **Committee Stage:** A detailed examination of the bill takes place, dealt with by the Public Bill Committee (which has supporters and opposition overlooking it). Amendments for discussion are selected by the chairman, and the committee votes on it. Any amendments made are published daily. Every clause in the bill must be agreed to, changed or removed before it is reprinted and on to the next stage.
- **Report Stage:** MPs on the floor of the house consider further amendments. All MPs may speak and vote, as well as all can suggest amendments or new clauses.
- **Third Reading:** Final chance for the house to debate the contents of the bill, where debate is limited to what is actually in the bill. Amendments can't be made. A vote is held on approving the bill. If passed, the bill is sent to the other house.

The bill must go through a very similar process in both the House of Commons and the House of Lords, until it finally proceeds with royal ascent, which enacts it as a law. We need to consider this process, because at each stage there are varying amounts of detail and information from different sources we can gather, limiting what we can know about a bill. If a bill is only on its first reading, it will not contain specific clauses, or details on commencement, which will make it more difficult to successfully summarise everything the bill proposes. This is where additional information such as the parliamentary debate may need to be accessed, which we could accomplish through Hansard^[126].

2.1.3 Legislation structure

Referring to the formatting and sections of the main textual body of a piece of legislation, understanding the structure is integral to interpreting it successfully. A legal text is irregular, not following general sentence or paragraph structures. The text is very specific and granular, often with clauses being long trailing sentences concatenated with 'and'.

- **Title:** Identifies its purpose and scope. It is a concise description of the bill.
- **Preamble:** Provides a brief introduction and explanation of the reasons for introducing the bill. Outlines the goals and objectives the bill aims to achieve.
- **Long Title:** More detailed version of the title, specifies the main purpose and intent of the bill.
- **Short Title:** Abbreviated name for the bill.
- **Clause-By-Clause Provisions:** Main body of the bill consists of a series of clauses each addressing a specific aspect of the proposed law. They are numbered sequentially. These clauses detail the changes or additions to existing legislation or lay out the new legal framework being proposed. The main body can break down into parts, chapters, sections (1, 2, 3, etc.), subsections ((1), (2), (3), etc.), paragraphs ((a), (b), (c), etc.) and subparagraphs ((i), (ii), (iii), etc.).
- **Schedules:** Attached to the bill and provide additional details such as lists, tables, or complex provisions that support the main clauses. Schedules are usually referred to by name, or number within clauses of the bill.

- **Enacting Formula:** Formal statement that indicates the authority under which the bill is being passed. It typically takes the form, "Be it enacted by the Queen's Most Excellent Majesty..."
- **Commencement:** Outlines when the different provisions of the bill may come into effect. Some may be immediately upon royal ascent, others might have a delayed commencement date.
- **Extent:** Defines geographical scope of the legislation, defining where in the UK the bill applies to.
- **Interpretation:** Section defines key terms used within the bill to avoid ambiguity or confusion.
- **Annotations:** Explanatory notes that provide additional context or clarification.

Both bills and laws follow a variation of this format, including or excluding elements as relevant to the document. Interpreting and breaking down this structure is the essence of this project, and how we will simplify our legislation.

As we are to be looking at text simplification, it is also important to understand what a given clause may look like. A clause often gives context in the form of an introductory sentence, and then lists a series of statements which build a run-on sentence.

(From Finance (No. 2) Act 2023[123])

"For the tax year 2023-24 the default rates of income tax are as follows—
(a)the default basic rate is 20%,
(b)the default higher rate is 40%, and
(c)the default additional rate is 45%."

Alternatively, the sentences may be completing the previous prompt such as:

(From Public Order Act 2023[129])

"The following persons are within this subsection—
(a)the undertaker;
(b)a person acting under the authority of the undertaker;
(c)a statutory undertaker;
(d)a person acting under the authority of a statutory undertaker."

2.2 Evaluation metrics

Whether optimising a model, or simply measuring success, we need methods to quantifiably evaluate specific aspects of a text. We will want to perform some level of human evaluation, using a large population of candidates to judge how successful our simplifications are[171], or using legal experts which can accurately validate the simplifications[40]. However, with our limited resources, we need additional approaches to interpret and analyse our results. This is why an extensive amount of research was conducted into numerical methods to interpret features of a text. These details serve an additional purpose, as we can use these metrics to filter out invalid answers, minimising inaccuracies or inadequate results. We may also use them to inform users - enhancing the transparency of our application[169].

2.2.1 Readability

Readability as it concerns this project, refers to the ease in which somebody can read a text. This inherently has an enormous number of factors, including vocabulary, topic, sentence structure and more. Not only does natural language change given time[151], but also where you are in the world, with regions having their own unique colloquialisms and vocabularies which may be foreign to someone who speaks the same language. Due to the disparities imposed by temporal or regional dialects, determining readability of a given text is always going to be a crude approximation. There is always going to be a degree of unknown error present. The assumption we make then is that every person who speaks a given language (for our purposes English) is empirically the same, and the only difference between them is the number of years of education they have had. For instance, we assume that they each share the same regional and temporal dialects, topic knowledge, inherent reading ability, etc. The incentive behind the assumption is that this allows us to look at purely quantitative measures of a text, and judge it off of those internal properties. We can implicitly calculate these properties, usually including word, character, syllable, and sentence frequency[31]. These intrinsic features of a text can then be used in some numerical measure, which describes how difficult a piece of text is to read. There are numerous readability metrics which

have been created, yet we need to identify the most relevant metrics, so we can best interpret our results. The result of most readability formulas is a USA reading grade. This correlates to the number of years of education an individual demonstrates. Considering this paper is targeted at the UK, this table(14) demonstrates the relationship between this readability score and UK education equivalents.

2.2.1.1 Flesch Kincaid grade level(FKGL)[78]

$$\text{Flesch Kincaid Grade Level} = 0.39 \left(\frac{\text{total words}}{\text{total sentences}} \right) + 11.8 \left(\frac{\text{total syllables}}{\text{total words}} \right) - 15.59$$

This reading metric was developed for the U.S. Navy in 1975, derived through testing a large sample of Navy personnel on their understanding of Navy passages. The formula takes the average sentence length, and the average number of syllables per word, and calculates a USA reading grade. This has since been used as a US military standard[98], and across the U.S. as a method to ensure official documents and contracts are under a certain threshold, so that they are accessible to the general public. Kincaid described the accuracy to be within one reading grade level[98], thus all uses must consider this error margin. Accuracy aside, the FKGL being so widely used, suggests it is an accurate reflection of the complexity of texts, especially being used in with contracts, which share many similarities to legal documents. However, we should consider the limitation that it stems from Navy personnel documents, which may bias the metric more in alternative contexts.

2.2.1.2 Flesch reading ease (FRE)[38]

$$\text{Flesch Reading Ease} = 206.835 - 1.015 \left(\frac{\text{total words}}{\text{total sentences}} \right) - 84.6 \left(\frac{\text{total syllables}}{\text{total words}} \right)$$

This metric was developed by Rudolf Flesch in 1940, who later assisted in the Navy's Flesch Kincaid grade level. This is one of the most widely used formulas, being built into tools such as Microsoft Office; and being used in legislation in Florida such that all insurance policies need a score of at least 45[88]. The formula returns a score from 0 to 100, where the higher the score, the easier it is to read(15). While this metric is widely used, it is very old, heavily impacting temporal dialect, but also was developed for improving newspaper readability[141], which can differ quite significantly to legal documents. Furthermore, Flesch himself stated that this method was to be quicker and easier to calculate, due to the lack of automation at the time[38].

2.2.1.3 Dale Chall readability (DC)[24]

$$\text{Dale Chall} = 0.1579 \left(\frac{\text{total difficult words}}{\text{total words}} \right) + 0.0496 \left(\frac{\text{total words}}{\text{total sentences}} \right) + c, \quad c = \begin{cases} 3.6365 & \frac{\text{total difficult words}}{\text{total words}} > 0.05 \\ 0 & \text{otherwise} \end{cases}$$

Inspired by Flesch-Kincaid readability, Edgar Dale and Jeanne Chall developed their formula in 1948, with a focus on education prior to college. Their formula returns a USA grade level. Their key difference is having a list of difficult words, determined by fourth grade students. This list was updated in 1995[20], with an updated formula.

2.2.1.4 Gunning Fog index (GF)[46]

$$\text{Gunning Fog index} = 0.4 \left[\left(\frac{\text{total words}}{\text{total sentences}} \right) + 100 \left(\frac{\text{total complex words}}{\text{total words}} \right) \right]$$

This metric, developed in 1952, has a similar approach to Dale Chall in looking at a more specific subset of complex words. Rather than compiling a list of words from participants, it instead defines a complex word as words with three or more syllables, excluding common suffixes. Additionally, this metric was developed to be fast, with the original algorithm only taking an extract of 100 words, and designed to be accessible for the time, with no equipment required[142]. Robert Gunning designed the algorithm based on the critique he received from his own paper's readability, with the formula favouring shorter, simpler sentences. Importantly, it was designed to identify complex research papers, targeting more difficult to read texts; notably being used by lawyers and other legal professionals[157]. This metric too returns a U.S. grade level.

2.2.1.5 Automated readability index (ARI)[164]

$$\text{ARI} = \left\lceil 4.71 \left(\frac{\text{total characters}}{\text{total words}} \right) + 0.5 \left(\frac{\text{total words}}{\text{total sentences}} \right) - 21.43 \right\rceil$$

ARI was developed in 1967, and also gives a U.S. grade level. This differentiates from most other metrics in that it uses the total characters as opposed to syllables to determine complex words. This was done as it was far faster to calculate, as a program could accurately count the number of characters, which then was used with electric typewriters. To make ARI compatible with our scores, we have to subtract 1.

2.2.1.6 Coleman Liau index (CLI)[23]

$$\text{Coleman Liau index} = 0.0588 \left(100 \frac{\text{total characters}}{\text{total words}} \right) - 0.296 \left(100 \frac{\text{total words}}{\text{total sentences}} \right) - 15.8$$

This method is very similar to ARI. Developed in 1975, returning a U.S. grade, and using characters instead of syllables, this was designed for similar reasons, trying to avoid the complexities of estimating the number of syllables in a given word.

2.2.1.7 Simple measure of goblegedook (SMOG)[52]

$$\text{SMOG} = 1.0430 \sqrt{\text{total polysyllables} \times \frac{30}{\text{total sentences}}} + 3.1291$$

As well as having the best name of all the readability metrics, SMOG is one of the most widely used metrics, particularly in the field of healthcare[55]. Giving us a U.S. grade, this was developed in 1969 as an improvement to the Gunning Fog index, trying to balance ease with accuracy. This measure has also been said to be a strong counterpart to the Flesch Kincaid formula, which underestimates reading difficulty compared to SMOG[36].

2.2.1.8 Evaluations

What can be noticed from each of these metrics, is that they all try to calculate reading difficulty from roughly the same abstract formula:

$$\text{Readability grade} = a \cdot \text{average sentence length} + b \cdot f(\text{total complex words}) + c$$

Such that $a, b, c \in \mathbb{R}$ and f is some function which interprets complex words. Throughout the formulas we see complex words being described by syllables, characters, or datasets, which each have limitations. The length of a word correlating to complexity is inherently naive, as discussed by the creators of ARI and CLI, who only used it for what was optimal for the time. Interpretations using syllables is an improvement, but there are a platitude of exceptions, such as ‘interestingly’. Finally, the database contains an immediate bias of the sample used to determine the database, the Dale Chall formula in particular not being as useful to us, as the database was built with fourth graders in mind. The average sentence length meanwhile is meant to account for sentence complexity, which similarly is naive, as a long sentence listing items, may be far simpler than a long sentence with complicated grammar, or contradictory remarks.

Each of these metrics share several common issues; they are outdated, context independent, and are order invariant. The majority of metrics used are from over 30 years ago, and stem from a variety of sources, subjecting them to temporal and regional dialect disparity[54]. Furthermore, these quantitative measures of text, crucially ignore the textual subject matter[71, 145]. Some topics and ideas will be inherently more difficult to understand, yet academic texts concerning complex theories would be given the same weighting as texts discussing mundane activities. The order invariance also impacts how we can use these metrics, as given a sentence, if we were to shuffle the words in any order, the metrics would still give the same result. This makes these metrics a poor judge of individual sentence structures.

It is important to consider how this may impact legal texts, to see if these metrics are appropriate for our task. As we have seen(2.1.3), legal sentences often take the form of run-on sentences which list items, or sentence completing clauses. This gives us multiple ways we can interpret what a ‘sentence’ is.

1. The sentence ends when it reaches the end of the clause, signified only by a full stop regardless of context.
2. Sentence completing clauses take the original sentence, concatenate the completion, and do this for every item that finishes the sentence, using all the results.
3. Sentences are divided by grammatical clause or ‘idea’, and sentences are split by new lines.

We chose the first method to use, as not only was this the easiest to implement, given our parsing structure, but the other two present unwanted inaccuracies. The second method is biased against sentence completion heavy texts, as it dramatically increases the number of sentences, when that isn’t representative of the text. The third method was originally used by the GF index, but since has been updated to use sentences for better performance. Using this run-on interpretation, it means that our legal texts will have fairly large average sentence lengths, generally making the legal texts be considered to be complex by these formulas.

The limitations of these metrics also imposed another restriction on how we could use them, in that they are not eligible for use as a training metric. With respect to machine learning (ML), and text simplification, it has been shown that the FKGL can artificially be modified to optimise[173], meaning using it as a metric to evaluate if a model is making successful actions is not appropriate. This can be abstracted to apply to all of these metrics, as they all are context independent and order invariant. As advised by Tanprasert and Kauchak[173], the metrics will only be used post hoc, as a tool to analyse the success of our simplifications.

While the metrics are flawed, they are the best estimate that we have for readability, and so are considered with a degree of error. The metrics we chose to focus on were FKGL, FRE, SMOG, GF and ARI. This was because of their wide usage, particularly in technical fields such as manuals or medicine, as well as how each of them categorise complex words in different ways, which will aid in interpreting our results. We have chosen to disregard Dale Chall readability, as well as the Raygor estimate[143] and Spache readability[168] for their limited scope beyond lower education. Coleman Liau was deemed unnecessary with the use of ARI.

2.2.2 Summarisation

While we can use the readability metrics to identify how difficult a text is to read, we need a method to evaluate how accurate our simplification is. To do this we can look at summarisation metrics[35], or consider semantic textual similarity (STS)[17]. Summarisation metrics are what have been used to train summarisation driven models, which aims to perform syntactic simplification(2.3.2), comparing the various n-grams between two pieces of text, to find a score reflecting precision and recall. On the other hand STS attempts to use cosine similarity (or other similarity measures) between text embeddings, to calculate how similar two pieces of text are, extrapolating meaning from that.

2.2.2.1 Recall-Oriented Understudy for Gisting Evaluation (ROUGE)[91]

$$\begin{aligned} \text{ROUGE-N}_{\text{recall}} &= \frac{|\text{original n-grams} \cap \text{summary n-grams}|}{|\text{original n-grams}|} \\ \text{ROUGE-N}_{\text{precision}} &= \frac{|\text{original n-grams} \cap \text{summary n-grams}|}{|\text{summary n-grams}|} \\ \text{ROUGE-N}_{\text{Fmeasure}} &= 2 \cdot \frac{\text{ROUGE-N}_{\text{recall}} \cdot \text{ROUGE-N}_{\text{precision}}}{\text{recall} + \text{precision}} \end{aligned}$$

This is a set of metrics to measure the overlap of n-grams between two strings. This is the most commonly used metric for evaluating text generation models such as summarisation and translation. ROUGE-1 considers unigrams, ROUGE-2 considers bigrams and so on. ROUGE-L is the longest common subsequence variant, looking at the longest sequence of shared words. Scores lie between 0 and 1, where a 1 suggests the two strings are identical. What is considered a good ROUGE score lies in a vague range dependent on the task[94, 27], which makes it harder for us to use. Additionally, because it only uses existing n-grams found within the original text, it means it is poor at evaluating non-extractive summarisation(2.3.3)[68].

2.2.2.2 Bilingual evaluation understudy (BLEU)[118]

$$\begin{aligned} \text{BLEU}_\omega &= BP \sum_{n=1}^{\infty} w_n \cdot \log p_n \\ p_n &= \frac{|\text{original n-grams} \cap \text{summary n-grams}|}{|\text{summary n-grams}|} \\ BP &= \begin{cases} 1 & |\text{summary}| > |\text{original}| \\ \exp\left(1 - \frac{|\text{original}|}{|\text{summary}|}\right) & \text{otherwise} \end{cases} \end{aligned}$$

This is a similar metric which expands on ROUGE, acting as its complement[85]. Instead of looking at n-grams individually, BLEU considers a geometric weighted sum of the precision of a summary. Additionally, it has a brevity penalty, which grants worse scores for summaries which are shorter than the original. This is a result of its original design for machine translation, as very short translations otherwise performed too successfully. Giving a score between 0 and 1, the higher the score, the better the translation[42]. However, for our purposes we will want to use BLEU to evaluate how successfully meaning has been carried across between the two texts, using Vashee’s scoring guidance[178]. Like ROUGE, there are several limitations[178], the most important of which being “no consideration of paraphrases or synonyms”.

2.2.2.3 Semantic textual similarity[65]

This approach uses word embeddings to mathematically compare the similarity between two texts. Word embeddings are word vectors represented in some continuous space, where similar meanings are close to each other. We can then use this embedding with our text, converting it into this vector. With two vectors we can take the cosine similarity[146] or Pearson correlation[65] between them and extract how similar in meaning two texts are. This is a powerful measure as it explores “different ways people might express the same idea”[17]. When using this metric, we need to find a suitable model, and there are inherent risks with how the model may have been trained[12].

2.2.2.4 BERTScore[189]

$$\begin{aligned} R_{\text{BERT}} &= \frac{1}{|x|} \sum_{x_j \in x} \max_{\hat{x}_i \in \hat{x}} (\mathbf{x}_i \cdot \hat{\mathbf{x}}_j) \\ P_{\text{BERT}} &= \frac{1}{|\hat{x}|} \sum_{x_j \in \hat{x}} \max_{\hat{x}_i \in x} (\mathbf{x}_i \cdot \hat{\mathbf{x}}_j) \\ F_{\text{BERT}} &= \frac{2P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \\ \text{s.t. } & x = \text{original}, \hat{x} = \text{summary} \end{aligned}$$

This metric is a specific case of STS, which uses the contextual embeddings from the BERT model, and crucially is designed to be a metric for text generation, with both a precision and recall score. The closer the score is to one, the more similar two texts are. Designed to be an improvement of BLEU, BERTScore is widely considered a reliable and capable method of calculating similarity between two inputs[2, 53]. It has been used as an improvement to ROUGE and BLEU, with a better correlation with human judgements[188]. While it has been shown to struggle in certain contexts[50], this metric will be our primary quantified measurement of how well our summaries retain their meaning. Helpfully, we can use the contextual embeddings generated by fine-tuned BERT models, which can be trained to the legal domain.

2.2.2.5 Evaluations

Due to the difficulty of judging how successfully meaning is carried between two texts, these metrics should be enough to derive a meaningful interpretation. However, we will need some level of human evaluation, where we use a small sample of summaries, and participants score how accurate a summary is. One additional issue we face is that we are unable to measure information lost, as there may be crucial pieces of information that aren’t retained in the summary. An approach we could take to account for this, is by adjusting the weighting of cosine similarity using TextRank[86, 101]. TextRank uses a given text to identify the most important sentences within itself, giving a score of self-referential similarity[10].

We can use this score as a weight for the most important sentences, which then evolves BERTScore to best consider the similarity between the most important parts of the text. While other metrics such as SARI[187] were considered, due to their similarities to worse measures such as ROUGE and BLEU, it seemed unnecessary.

2.2.3 Orthographic and syntactic

One limitation commonly discussed between readability and summarisation metrics, is their inability to capture if a text is orthographically and syntactically correct[54]. This refers to cohering to the spelling conventions and syntax of a language. This is important as if we get responses which do not make sense, are incoherent, or contain garbled strings of characters, then we need to be able to flag these texts as incorrect. Fortunately, unlike other metrics, these can be quantified by the number of rule breaks that are found. Creating a system to identify rule breaks would be time-consuming, thus we will be using an existing implementation[103]. We will need to ensure only significant rules are being broken, as some tools will consider unimportant errors such as using English spellings.

2.2.4 Bias

As detailed in our requirements(1.3.2), we need to be able to evaluate if our application is biased. AI has been demonstrated to be biased in a multitude of examples[25, 26], stemming from the training data or the training process[176]. As we plan to use a LLM, we will have to decide on a LLM that has displayed minimal bias, but that alone isn't enough. We need to enforce some safeguard which can aid us in concluding if our outputs are appropriate to display. Detecting the presence of bias in a text is difficult, as not only is bias never clearly stated, but even humans struggle to detect bias[144], especially their own biases[32]. Furthermore, text generation precludes the use of many standard fairness metrics[96] due to its qualitative nature. Our task adds additional constraints in that the legal domain is inherently biased. Implicit bias[170, 72] is present at all stages of the legal process, and it is impossible to avoid in a legislative document discussing what laws are or aren't in place. While the UK government protects groups from discrimination[43], that doesn't prevent bias being found in legislation, with inherent legal differences existing between groups[7]. Legislation concerning homosexuality offences[130] showcases bias both towards and against the homosexual population in their texts; and the 2010 Equality Act[121] discusses how specific minority groups should be protected. Both positive and negative bias are present throughout legal texts, intentionally or otherwise. This means using bias identifying models will not be particularly helpful. An alternative we can use is sentiment, which reflects the tone of the text. The tone of our summary in of itself isn't useful, as the topic which we may be summarising may be overtly positive or negative. However, we can use the sentiment analysis of both the original text and summarised text, and compare them to see if they are similar. If the sentiment of the original text is very positive and the sentiment of the summary is very negative, that implies there must be some bias impacting the result. Using this method, we can identify an error threshold, in which the response is appropriate. Sentiment analysis of a text can be calculated through a variety of models accessed through HuggingFace[132], which we will need to decide what is the most appropriate model to use.

2.2.5 Information theory

$$H(X) = - \sum_{i=1}^n p(x_i) \log(p(x_i)), \quad p(x_i) = \text{probability of token } x_i$$

Entropy is an alternative method we can use to evaluate how readable a text is. Under linguistic information theory developed by Shannon[161], entropy measures the randomness in a given text, which is how surprising an outcome may be. With respect to texts, it uses the probability of a token occurring, and it has been shown that texts of a high entropy are implicitly complex[149], or incoherent. For example English has been shown to constantly give lower entropy than sentences composed of random words, because English follows a set of rules. With this, we can extract another measure of readability, in the form of comprehension, as well as possibly determining an entropy limit, a threshold that if surpassed, the text is fundamentally illegible.

2.3 Text simplification

Our aims detail the need for text simplification, an automated process to take a string of any length, and make the text easier to read while retaining its original meaning. This comprises multiple methods we

need to combine. We can simplify a text both lexically and syntactically as detailed by Siddharthan[162], and we can summarise a text, which reduces the total size[67]. Summarisation can either be extractive or abstractive[39, 97]. Extractive summarisation aims to isolate the most important and relevant quotes from a text, pulling them out into a summary. Abstractive summarisation aims to rewrite the key topics into a coherent self-contained passage, trying to best reflect the meaning of the original text, while rearranging or changing the text. Abstractive summarisation is very akin to simplification, and is a non-trivial task. We need to understand each simplification method in isolation, so we can best understand how we can combine them meaningfully.

2.3.1 Lexical simplification

One aspect of complex texts is the abundance of complex words. Lexical simplification aims to replace complex words with simpler corresponding terms. Complex words can be defined in a multitude of ways, but linguistics states that a word with two or more morphemes, are complex[30], where morphemes can be understood as the smallest meaning-bearing unit words. As we have seen with readability metrics, other interpretations of complex words are dependent on the number of characters, the number of syllables, or simply a dataset of difficult words. Another measure we could use is frequency, where words which appear less frequently in texts are more likely to be ‘complex’[140, 152]. We can access a representation of this information with the English Word Frequency dataset[174] or the google-10000-english dataset[74], which are both derived from the Google Web Trillion Word corpus. the size of the corpus used to generate them make them representative of real world word frequencies.

Using one of the suggested methods of complex word analysis, lexical simplification is then achieved given a dataset of word synonyms such as WordNet[177]. The most complex words are identified, selected and replaced. This can be completed greedily, where we rank the words of a given sentence by complexity, then choose the top k words to replace with the least complex synonym candidates. The caution with the greedy method is the loss of context, especially with respect to domain specific knowledge, as certain legal phrases - which we can identify using the UK parliamentary glossary[125] - may contain words which are complex, but do not make sense out of context to replace. This is where methods such as LS-BERT[137] may be a strong alternative, considering the context of a given sentence. Despite this, LS-BERT, and other lexical simplification models, only consider single words as opposed to phrases, and when we consider how much of the glossary consists of phrases, it may have limited effectiveness.

2.3.2 Syntactic simplification

Another aspect of complex texts is the syntax, the way that information is delivered to the reader. In an isolated sentence, the syntax is complex if there is the presence of one independent clause, and one or more dependent clauses[165]. However, we can expand this to a broader complex syntax if we consider combining multiple independent clauses, creating complex compound sentences. In legal texts, we have seen they are full of long complex sentences which list dependent clauses; as well as compounding independent statements together(2.1.3). Complexity can exist beyond just clauses, with grammatical complexity and the order of how information may be worded.

Syntactic simplification aims to reduce the complexity of individual sentences, by combining clauses, removing unnecessary information and reducing grammatical complexity, while retaining its meaning[162]. A common method used is sentence simplification across all sentences in a text, with existing models using a variety of corpuses[159]. However, most are limited to single sentence simplification, where they can only simplify information in an isolated context. This isn’t useful for our project, as due to the long length of legislation, and how often many ideas are repeated slightly differently between legal clauses, simplifying only sentences is not appropriate for our goal. We could use extractive summarisation techniques such as TextRank, to isolate the most important sentences, and then simplify them individually before concatenating them back together. This method does come with the limitation of lacking surrounding context, such as what comes before and after that sentence in the text.

2.3.3 Extractive summarisation

One of the key elements that makes legal texts so difficult to read is how long and verbose they are. Therefore, we want to use summarisation to make the texts shorter to read, and more digestible. Extractive summarisation does this by extracting the most important topics or sentences, ripping direct lines

from the text, as this will best retain its meaning[97]. This can be accomplished in several methods, such as TextRank[101], TF-IDF[154] or SVM[58]. Studies seem to suggest that TextRank is the most successful of these techniques, especially in an unsupervised context[139].

2.3.4 Abstractive summarisation

This is closer to how humans approach summarisation, trying to capture the underlying ideas of the text[67]. It relies on paraphrasing, and finding easier ways to express the contents of the document. Compared to extractive summarisation which is like a highlighter, abstractive summarisation acts like a pen[135]. This can be done as a combination of extractive summarisation, sentence simplification and lexical simplification; or it can be achieved with transformer models. These are models which can generate text, and can have been specifically trained for summarisation like certain models on HuggingFace[64]. Alternatively they can be an LLM, prompted to summarise given text. Abstractive summarisation on legal texts is what we are striving to achieve, and how we can best optimise our inputs and outputs with transformer models, to give us the best simplified summary.

2.4 Large language models

Since the introduction of the transformer in 2017[179] there has been numerous developments in generative AI. Transformers modified neural networks to utilise self-attention, weighing the importance of each word with respect to every other token[160], which allows for long signals, such as large text blocks, to retain context in generation. This has been largely used with language models like GPT[114] and BERT[28]. Large language models, as the name suggests, use enormous neural networks with millions to trillions of parameters, which refer to the weights and biases of the network. With an exponential increase in the number of parameters used to train models[163], it has allowed many generative AIs to flourish, which can excel at many NLP tasks.

2.4.1 Emergent abilities in LLMs

While many generative language models were initially designed to predict what text would follow given a prompt, models trained on more parameters have demonstrated emergent abilities[184]. The rate at which abilities improve “are at incredibly rapid paces and at unpredictable scales”[110]. This is what inspired this project, with summarisation being a leading use of LLMs, demonstrated by how it is advertised for commercial uses in summarising articles and research papers for students[4, 93, 100]. These summarisation capabilities have already been used in finance[59], medicine[90], and in law(2.5). The successes found in these projects have exemplified the summarisation capabilities in a complex field. We can understand the success of summarisation because of the contextual knowledge that an LLM has been provided with about specific subject matters, as seen with FinBERT[59], where LLMs trained with finance specific knowledge performed better than those without. Moreover LLMs are able to generate convincing human text, and generally can identify what text is considered to be more ‘simple’.

What separates LLMs from other methods is the ability to generate new text effectively. Abstractive summarisation methods in the past were held back by the linguistic quality that wasn’t found before the transformer architecture was introduced[76]. LLMs excel at producing quality written responses, and we will be able to leverage this with other text simplification techniques. Furthermore, LLMs are able to be fine-tuned to better perform on specific tasks, such as summarisation. Summarisation is an interesting challenge, as LLMs are great at abstracting away irrelevant information, such as meaningless references and citations (useful for legislation), which are not important for a summary. This data abstraction does bring a loss of accuracy, which is difficult to control and monitor, resulting in certain ethical concerns[102, 117]. Additionally, LLMs do have a limited context window, meaning inputs with too many tokens will not work. We will still need an approach to parsing long texts.

2.4.2 LLM selection

As we are not trying to build a LLM from the ground up, we need to select the most appropriate one to use. There were many candidates, so we immediately reduced our selection to models with parameters in the billions, as they were the most recent and most successful models[75]. Additionally, we wanted our model to be open source, partially for the increased control we have in using the model, and partially so that we could use it for free; crucially ruling out GPT-3 and GPT-4[114]. Our next criteria was to look

at context window size, as this reduces our dependency on long text methods, and will improve single summaries. Moreover, we considered the number of parameters of the model, which was a good indication of approximate performance. Another metric we looked closely at was the WinoGrande score[153] which tests an AI’s commonsense reasoning. While there are many measures which we can use to evaluate a model[21, 56], WinoGrande was given priority as it tested logical inference as opposed to academic abilities. Since we care more about logically structuring and summarising from a given document, inherent academic information is less important. Other miscellaneous features were considered as well, such as the amount of support and documentation, as well as its general acclaim and usage. We wanted to explore the datasets that were used to train the LLMs to detect bias, however the majority of companies do not disclose their training data.

Our LLM we selected to use was Llama2, with either 7b or 13b parameters. As the number of parameters increase, so do the system requirements, as well as the time taken to execute. This is why we didn’t use the 70 billion parameter model, especially since we are simply trying to build the framework. We selected Llama2 as it met all the preferred requirements while being inexpensive to run, with several standout traits that made it desirable. Firstly, it demonstrated strong factual correctness in summarisation tasks with respect to GPT-3.5 and GPT-4[138]. Next, Llama2 has been shown to be effective at summarising U.S. bills. While not being as good as GPT-4, it is on par with legislative interns; and was shown to improve in results with careful prompt engineering[113]. Finally, Llama2 has extensive documentation due to how accessible it is to more devices (costing 3% of what GPT-4 costs[113]); as well as being integrated with both HuggingFace[155] and LangChain[83]. Being open source, well documented and a compromise of cost and ability, Llama2 was the best choice of model to work with.

2.4.3 LLM prompting

With the model decided, we need to know how to best prompt it. A LLM is a generative model, where it has been trained to generate the most likely text to come after a series of tokens, which means depending on the use of a specific token it could dramatically change the text sequence we retrieve. Firstly there are six key tokens we need to understand:

Token	Meaning
<s>, <\s>	Defines the beginning and end of a sequence.
[INST], [\INST]	Defines the beginning and end of an instruction.
<<SYS>>, <<\SYS>>	Defines the beginning and end of a system message.

Table 2: Key Llama2 tokens.

Llama2 prompts were trained to follow an instruction-response pattern. An instruction is given, delimited by the [INST] tokens, and then a response is provided[13]. The instruction and response are entirely contained within a sequence. The system message is a priority message given to the AI which describes the role and premise of the AI, as well as letting us define global instructions to prioritise. When we prompt the model, we are allowed to write any of these tokens, which means we can not only define the system messages, but we can also define the beginning of the response output, which gives us control on how the response is given(1)[13, 48].

Additionally, we can specify the specific format the model should return information, with existing formats like lists and JSON. We can improve how accurately a format is followed by using one-shot learning[133], providing a single example sequence which can massively improve the performance of the AI[82, 13]. We can additionally use chain of thought prompting, by using the output from a previous prompt.

```

1 <s> [INST] <<SYS>>
2 Your objective is to simplify and provide an overview a given piece of
3 UK legislation, capturing the most important impacts and information of
4 the legislation.
5 You are to make your responses as easy to understand and accessible as
6 possible, using minimal specialist vocabulary.
7 Please ensure your responses are unbiased and never share false
8 information.
9 If a question does not make any sense, or is not factually coherent then
10 do not answer.
11 <</SYS>>
12 Given is the title and long title of a piece of UK legislation. Explain
13 what this legislation does in very simple terms. Make it very easy to
14 understand what it is doing.
15 ```TITLE: {TITLE}
16 LONG TITLE: {LONGTITLE}``` [/INST]
17 SUMMARY:

```

Figure 1: Depiction of an example prompt to Llama2.

The other aspect of prompting we need to consider are the configuration hyperparameters, which give us access to a broad range of ways we can influence the results. All the hyperparameters are listed in the generation config provided by HuggingFace[60].

Hyperparameters	Type	Impacts
max_new_tokens, min_new_tokens	int	Determines the min and max tokens, excluding our prompt, will be generated. Will need to be optimised with the context window so that responses don't leave it.
do_sample	bool	If false, the model will use deterministic greedy decoding, which will always choose the most probable token next. If true, the model will use stochastic sampling, which samples across a probability distribution, thus making answers more random.
temperature	float: [0,1]	Only applies when do_sample is true. Determines how diverse and random our responses will be, prompting more creative responses. Low temperatures can be very robotic[69] which could make a text less readable for audiences, but we want to minimise creativity to avoid risk of speculation.
top_p, top_k	float: [0,1] int	These refer to two different methods of filtering while we sample probabilities. We take the top k probable tokens, and then keep a percentage p of those tokens. Keeping these values reduce variety, and focus the model's response[104].
diversity_penalty, repetition_penalty, length_penalty	float	Each of these influence the response generated, changing the respective factors of the text. Optimising these will be key for ideal results.

Table 3: Most important generation hyperparameters and their effects.

2.4.4 Chunking problem

LLMs work by using a long term memory which operates in the range of the model's context window. Every token in the context window is considered when generating the new token, however anything outside the context window is lost and the LLM is not able to use it. While larger context windows are ideal, there is no LLM which has a context window able to encapsulate the larger legal documents we are facing. To summarise larger documents which exceed the context window, we need to chunk the text, dividing it into smaller 'chunks' which can then be individually summarised. There are multiple methods of chunking we consider, to determine the most appropriate to use.

There are several basic chunking methods, such as fixed size chunking (FSC) which takes a set number of tokens each time; and content defined chunking (CDC) which chunks dependent on tokens or symbols[108]. There is also the sliding window variant of these, which has a level of overlap between chunks, which allows for some context and continuity between the chunks[33]. Your final summary can then be the concatenation of the summary of each chunk; or we can perform recursive summarisation, and summarise the respective summaries until we encapsulate every chunk[156]. Each of these methods bring several limitations. FSC can cut off words or sentences, which makes them get overlooked in summaries. CDC avoids this by having full sentences, but sentences may lose context due to the greedy nature of this chunking. Sliding window increases coherence between the chunks, but at the sacrifice of increased data duplication. Data duplication results in unnecessarily long concatenated summaries, and can heavily skew recursive summarisation into focusing on that data if it is not removed. While we can try to extract duplicated data, this is inherently a difficult task due to our use of abstractive summarisation, therefore tokens may heavily differ while meaning remains the same. Recursive summarisation brings additional risks of LLM hallucinations[183, 29], information which it has generated completely on its own. By prompting a LLM repeatedly it increases the chance of hallucinations appearing, simply due to having more blanks to fill.

Our domain of legislation is unique in having a very defined structure, which means we can expand CDC and instead of chunking at tokens, we chunk at entire sections. Once we have a series of chunks and their summaries, we can retain and concatenate them for a low level understanding of a section of the document. We will also want to find some method of combining all the subsequent summaries. Using an improved recursive summarisation, we can create summaries of sections at multiple levels of detail, and eventually use that for a final summary. We will need to experiment and find methods to reduce hallucinations.

2.5 Existing solutions

Trying to summarise legislation isn't a new task, and each have used differing ML approaches. Firstly is Padolsey's approach of using LLMs to parse and simplify UK bills[117]. This is trying to perform a very similar task, using the emergent abilities in LLMs to simplify bills, which makes it a great baseline to compare how successful my project is. Their implementation has several limitations, particularly being constrained to specific bills, with not much in place of an automated system to access legal texts. Additionally, there were common occurrences of hallucinations present in the text, with inaccuracies appearing. They also used a naive chunking method(2.4.4) which may lose context throughout the summarisation process. We will be able to use our metrics on Padolsey's texts to compare our results.

There have been other solutions which attempt to summarise more general legal texts, including documents only pertaining to law or contracts, where we see several issues we may need to overcome. Garimella's paper suggests that lexical simplification methods struggle due to a lack of a legal lexicon, occasionally generating replacements which are not appropriate[40]. This is compounded by domain specific language models seeming to perform better than their alternatives[18].

3 Implementation

3.1 Design

The entire problem was broken down into several succinct modules which could each be constructed and worked on individually(4). The increased modularity allowed for an easy to assemble pipeline which could have components changed and edited without heavily impacting the rest. Each core module generally corresponded to a requirement, and were developed to have consistent inputs and outputs. These abstracted inputs and outputs form the basis of our sequential pipeline. To understand the processing of the implementation, we can visualise the pipeline(2) to see how each module follows from each other.

Modules	Inputs		Outputs		Objective
	Value	Type	Value	Type	
Bill acquisition	Bill ID	int	Legal document in HTML or PDF format	str	We prompt the Bills API to recover given information about the bill.
Act acquisition	Year and ID	(int, int)	Legal document in HTML or PDF format	str	We prompt the Legislation API to recover given information about the act.
Parsing	HTML or PDF formatted legal text	str	JSON formatted legal text	dict()	We convert the legal document into a JSON, sectioned by legislation structure (2.1.3).
Chunking	JSON formatted text	dict()	Array of logically divided chunks and their token lengths	[dict()]	We divide up the JSON into respective sections using token limits.
Generation	Array of string chunks	[dict()]	Different summaries generated using the chunks	dict()	We perform our abstractive summarisation and create our different summaries.
Text metrics	Original text and summary texts	dict()	All relevant metrics and their corresponding values	[dict()]	We gather all relevant scores to analyse.
Interface	Summary texts and text metrics	Legislation	Webpage displaying information	HTML page	We simply display all relevant pieces of data.

Table 4: Every module in the pipeline.

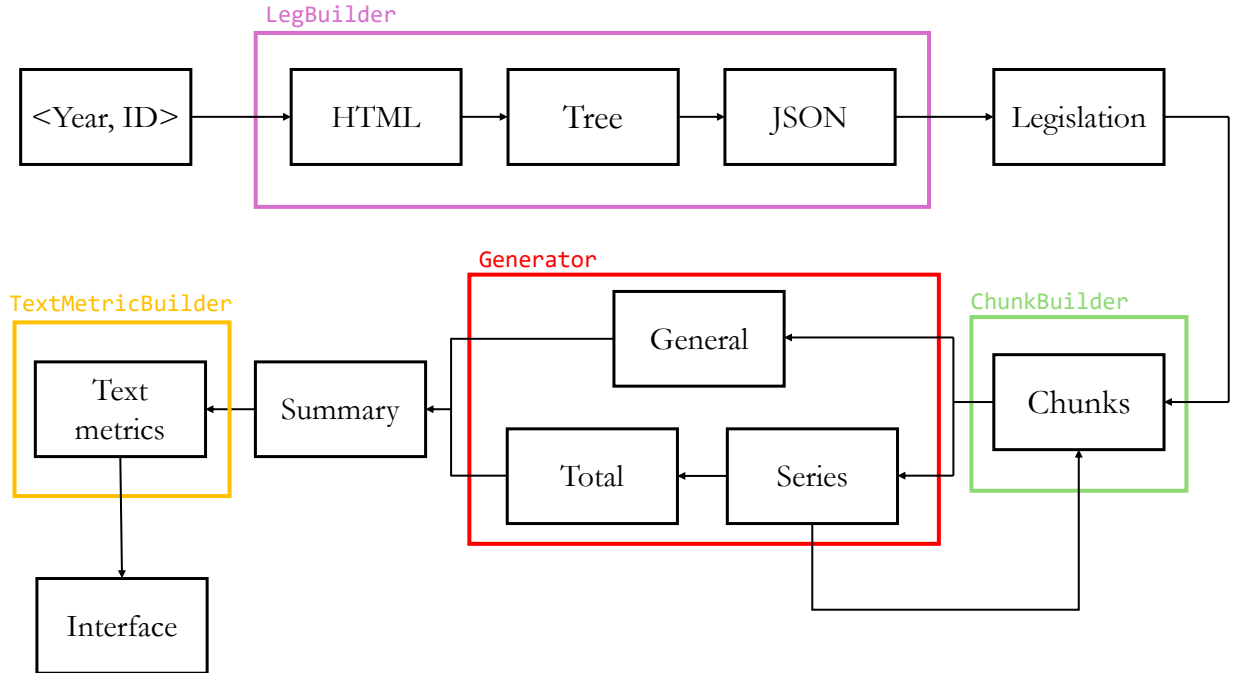


Figure 2: Abstracted legislation simplification pipeline.

Due to the nature of modules requiring inputs from other parts of the pipeline, the actual program is slightly more complex. We can see the more complex dependencies between modules demonstrated in the class diagram(3). The pipeline passes a Legislation object through it until all attributes are assigned a value. A list of these objects can be used by any interface implementation. The original text is only stored for ease of testing; but if the app were to be deployed it would be removed for space efficiency. The design of the pipeline constantly building and adding to the legislation object was done so that new attributes could be easily added to the legislation, which could immediately be accessed by the interface.

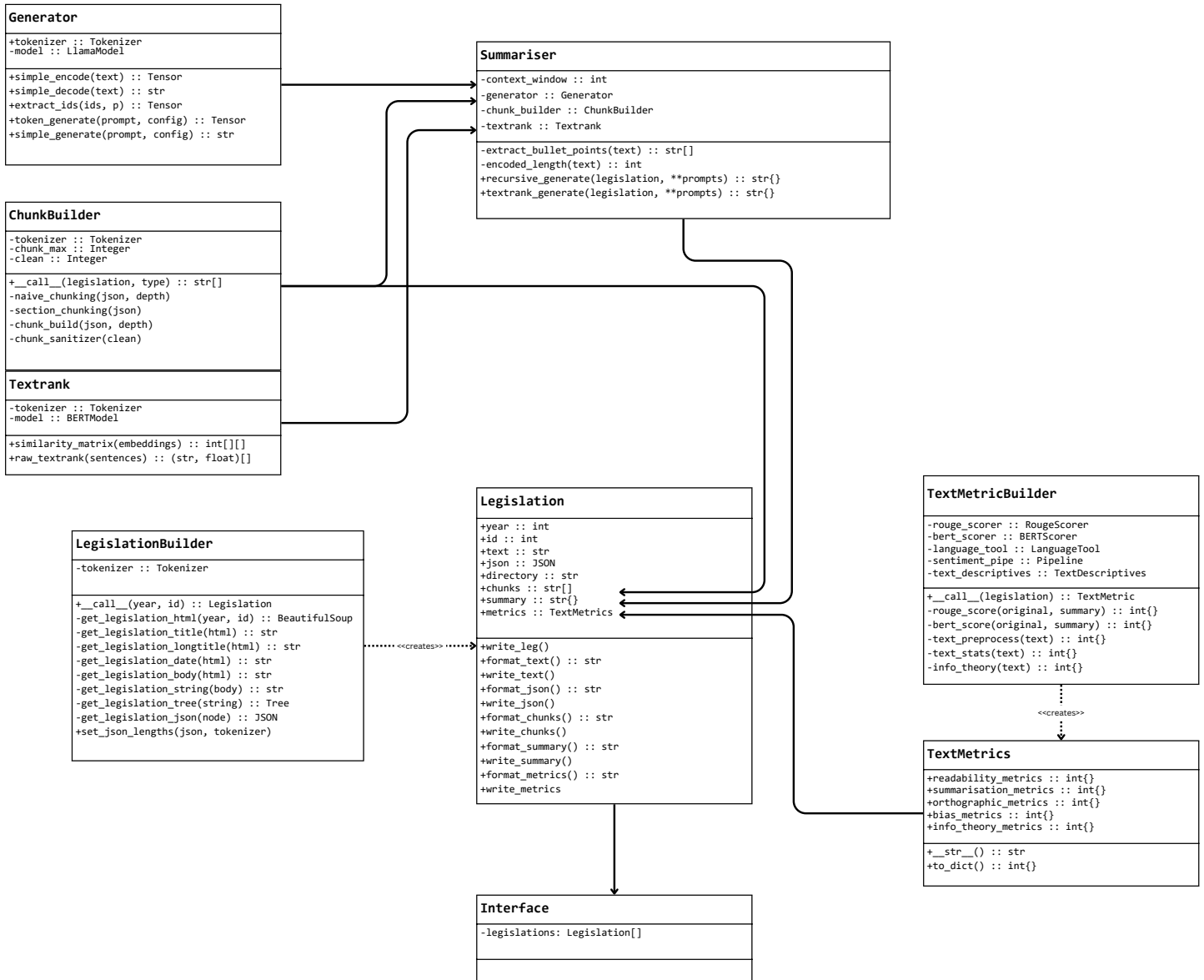


Figure 3: Class diagram of the pipeline implementation.

Developing each module was completed in Python, for the useful libraries it has access to. This included HuggingFace, the AI NLP hub we would be accessing many models from. Additionally, Python was the best documented for using Llama2, and other ML tools. The simplicity of Python and its comprehensive documentation, let us focus on implementation as opposed to learning how to use the language. Furthermore, the drawbacks of Python were less influential in this project. The slower speed compared to other languages was mitigated by the reliance on CUDA for intensive operations, and the lack of strong

typing has far less impact on a single-person team.

3.2 Bill and Legislation APIs

We first needed to access the documents we were to summarise. Both bills and acts can be accessed using a Parliament provided API.

3.2.1 Legislation API

Passed acts can be very easily accessed. The website <https://www.legislation.gov.uk/> provides access to all enacted legislation from between 1988 to today[87]. We can access each legislation using a composite key of the year it was enacted, and its ID - the order in which the act was released that year. Using the following URL,

```
"https://www.legislation.gov.uk/ukpga/year/id/enacted?view=plain"
```

we can access and retrieve a raw text HTML document detailing a given piece of legislation. This can be easily navigated for its key components, such as title, longtitle, body, etc. by using BeautifulSoup[150].

3.2.2 Bills API

Bills are more difficult. We can access a list of all bills using their API[120], which has a large amount of customisation and search options. However, the legal texts aren't contained within this API call. Instead, we need the bill ID which we use to query their publications, which each have a publication ID. A publication may have a link to access the document, or they may have a list of files which contain a document ID. You would then use a publication ID and document ID to query and download a file. When searching for the publication type that we want, we would be looking for a publication of type 5, which is the legal text bill. When we are given a file we can download, we can access its direct bytes content, and interpret it to raw text using PyMuPDF[167]. However, if there are only links in the publication, we cannot access its direct HTML content, as we get a 403 status code. This is an unfortunate result of the link being protected from bots, and so the only way to access the data without permissions is through Selenium web scraping[66].

Once we have gotten our raw text, our real problem arises. Each text acquired with a different method had a different text structure. We found at least three structures with the text, depending on if it was a file, or on the type of link stored. Since we didn't want our project to be purely developing parsers for these texts, we opted to focus on acts from the Legislation API. This was because they were consistent in their structure, and far easier to acquire. Bills were left as an extension for future work.

3.3 Parsing

Using BeautifulSoup to access the body element, we are able to go and access the key structure of the legislation. We can use `html2text`[172] to abstract out the raw information, and similar to Padolsey[117] we can greedily chunk the legislation into the LLM. However, the limitation here is the loss of context, where we can no longer summarise sections or blocks of the original text and instead summarise an overlap of information. This not only makes it more difficult to add multiple levels of simplification, but also may result in a loss of information when looking at a given chunk. Because legal documents follow a more rigid structure, by being able to break down our text into logical divisions, we may be able to extract more meaningful summarisations from further experimentation. Therefore, we aimed to convert the document into sections defined by the structure, using a JSON format. Child elements would be divisions of a parent section, such as how parts are divided into chapters, chapters into sections, etc.

The raw HTML file denotes each indentation level as a series of tags, which has an associated style sheet class. As opposed to using child elements, the indentation level (and subsequently the section of the text) is determined by the style sheet, which we can access through the website. If the documents only consisted of the same key structural elements identified, this would be fairly easy to extract out, however throughout the documents are amendments to prior legal texts, which in turn describes an enormous variety of divisions. These amendments are declared using quotes, and uses a slice of the original document it is amending over. We cannot use the prefixes that were previously discussed[44] because of these

amendments altering how they would be embedded into the JSON. One option was to scourer through the different classes and devise an appropriate algorithm to make the most accurate JSON formatted legislation, but since this was not the focus of this project, we instead used a ‘good enough’ method. The most important features we wanted to identify were the different sections, which could be easily achieved by using `html2text`. Due to a quirk in how legislation HTML is written, it meant that for the important sections we wanted to find, there were easy to describe identifiers at the start of lines.

Envisioning the lines of text as a tree that had been flattened(4), we took the structure of a legislation and related each depth level to a different meaning. The leaves would be the clauses, and the title would act as the root of the document. This gave us several options for how we could use the text when summarising. To convert the text to the tree shape, we associated a depth to each line, using regex matches to determine what type of line it is. Then we used the depth to determine where to position the node, shown in the following pseudocode(1). As the text is ordered, we can always raise our depth to the appropriate level when encountering a higher depth value. We then fill our tree nodes with respective information, such as the text, number of tokens, the type and the relevant prefix.

Algorithm 1: Text to tree

Input: Text

Output: Tree root

```

root ← TreeNode(0) true_root ← root
for line ∈ text do
  local_depth ← depth(line) new_root ← TreeNode(0)
  if local_depth > root.depth then
    | root.children ← root.children + [new_root]
  else if local_depth == root.depth then
    | root.parent.children ← root.parent.children + [new_root]
  else
    | while root.depth ≠ local_depth - 1 do
    |   | root ← root.parent;
    |   | root.children ← root.children + [new_root]
  | root ← new_root
return true_root

```

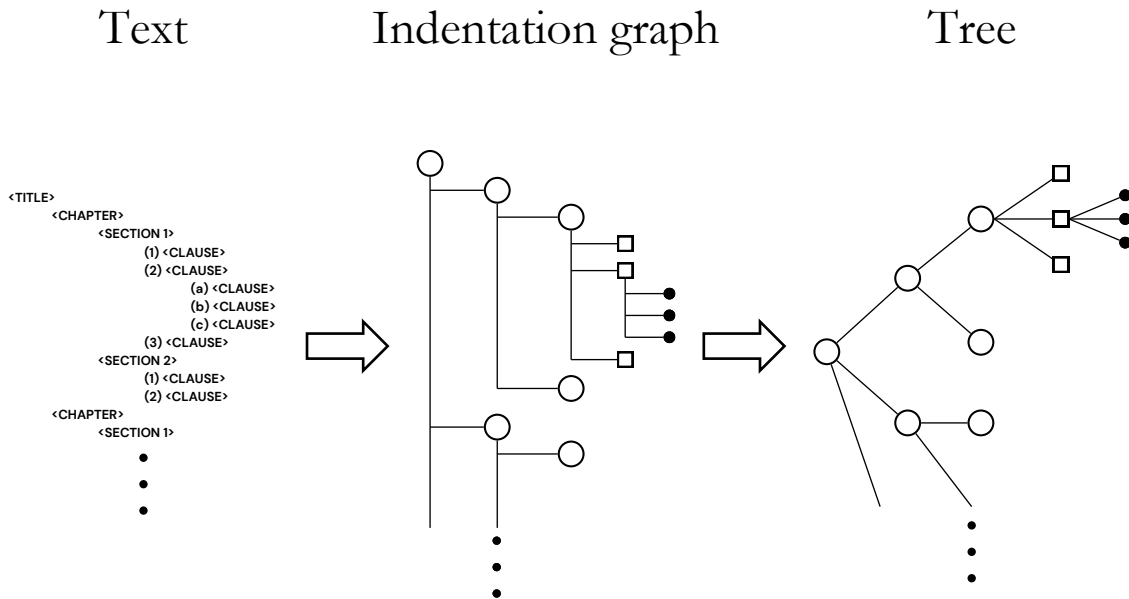


Figure 4: Visualisation of the parsing of a legal text file to a tree.

As we parsed over each line, we could perform additional sanitization operations on the text, such as trying to remove URLs found in the text. We defined an alternate stopping condition as well, where if a schedule was encountered, we would stop parsing. This was done as schedules contained a lot of more technical details on the act’s implementation, rather than what the legislation does, only serving to make the documents even longer. Therefore, we decided to truncate out schedule sections for efficiency and more effective summaries. Once we have this tree object, we then can recurse over it systematically and produce a JSON object with which we can provide some additional details useful for chunking, such as text, section and sequential token counts. We convert it to a JSON for storage and testing purposes, however in a more streamlined system this may not be necessary.

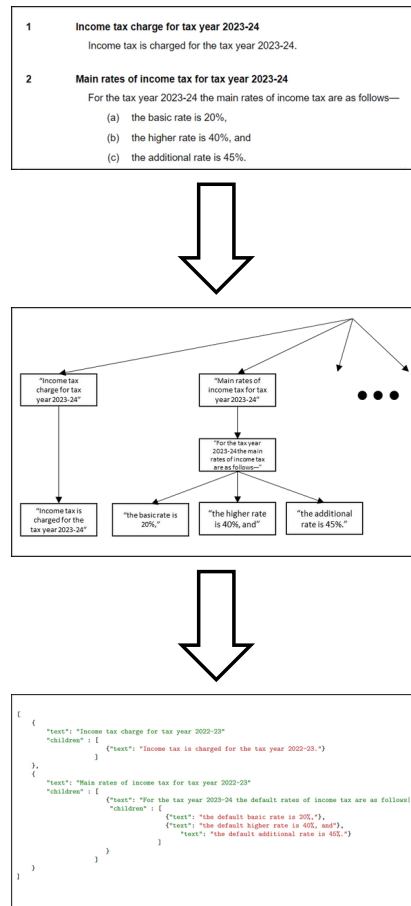


Figure 5: Clauses to Tree to JSON example visualisation.

3.4 Chunking

With our JSON defined, all relevant details are stored inside the attributes to be able to chunk using our desired method. We always ensure that we use a full sentence/clause, as this way we reduce the semantic coherence loss. Our chunker takes a maximum number of tokens it can take, and a ‘clean’ value, which defines how many tokens a chunk must have at minimum. We define two chunking techniques, naive and sectioned. Naive chunking has us go sequentially through our JSON, tracking the total number of tokens, and if we can add the next line to the chunk without exceeding the maximum; acting as a preorder traversal of the JSON. Sectioned chunking instead looks at every child of the current section they are in. If the total tokens present in that child is less than the max, then we naively chunk that child together. Else, we simply recurse to investigate the children of that tree to chunk them.

This module is designed to allow for some additional useful features of chunking, importantly defining a start sequence, where we only chunk after some given conditions are met. We decided it was only worth chunking after the text containing the enactment(2.1.3) as that introductory spiel not only isn't unique, but also serves as additional bloat to the text, which would be a waste of tokens for the LLM. The chunker also 'cleans' the chunks, deleting any under a certain threshold, with the assumption that some chunks will be too small to hold meaningful data, making it more likely for an LLM to hallucinate with that chunk.

3.5 Generation

This is the core of the pipeline, taking in our chunks and producing several relevant summaries from a given prompt and config. We designed our generator to take prompts and configs as inputs, so we could learn the ideal hyperparameters with experimentation.

3.5.1 Model loading

Generation could be achieved in several methods, either using a higher level approach which keeps the process abstracted, or working lower level to directly parsing in and out the tokens. Initially an abstracted pipeline was created using HuggingFace and LangChain[83] as it was at first easier to design and use. Later in the project we pivoted to using the model directly, allowing for better control over generation. Experimenting with Llama2 models[99], we decided upon using the 13 billion parameter model consistently, as 7b often gave worse responses, and 70b exceeded memory available. We load the model and its tokenizer using HuggingFace, with a specified quantization config. Quantization reduces memory usage with lower precision data types[63], which lets us actually load the model into memory, as well as massively improving speed. We also ensure we load our program with CUDA, so that we can optimise usage across the GPUs available. When using CUDA, we must also convert our tokens to appropriate CUDA values.

With our model loaded we can simply call it with a tensor of tokens which the tokenizer produces. We also unpack a config into our model impacting generation. As we want to be able to experiment with prompts and hyperparameter values, we streamline the generator to be able to take all of these as parameters. Using the lower level tokenization means that the tokens generated by the model include the original prompt, which we extract using the length of the given prompt.

3.5.2 Context window

Wrapped around our Generator class, we now define our Summariser, which uses our chunker and our generator. Our primary generate function takes the legislation object, and all relevant prompts and configs for every time we use the model. Before we can call the chunker, we need to determine what is the maximum size of a chunk that we allow (maximum number of possible tokens in a single chunk).

$$|title| + |longtitle| + |prompt| + \text{max chunk size} + \text{max new tokens} < \text{Context Window}$$

$$|prompt| + \text{max chunk size} + \text{max new tokens} < 4096 - c$$

The context window consists of the given prompt (of which the majority of prompts include the title and longtitle, hence they are included as a constant c), the length of the chunk to summarise and the maximum number of new tokens that can be generated. We want to try to always stay in this range. While LLMs can generate beyond a context window, by using context from its previous tokens, this is not beneficial as all information we want it to glean is inside the original window. A chunk should fit to the size of the prompt and max new tokens, so if we can solve what max new tokens should be, we can determine max chunk size. The max new tokens can be defined in a config, setting a hard cap on generation. If too low, it can cut off the LLM response mid-sentence. Therefore, we want the max new tokens to always be adequately high enough to summarise the chunk. However, the higher the max new tokens, the smaller the average chunk will be, and thus the less that our summary will produce. We don't want small chunks as they are inefficient to process and have a higher chance of hallucinations. This means we have to ensure we make the max new tokens as small as possible without cutting off the LLM. Due to the qualitative nature of this equation, we can't determine it outright, only estimate. By running differently sized inputs through our model, and extracting the maximum length of the chunk,

the largest chunk generation we found was 782 tokens. As this wasn't exhaustive, we rounded upwards, and concluded the majority of all chunk summaries will be under 1000 tokens, therefore acting as our lowerbound.

$$\text{max chunk size} = \lfloor 4096 - c - |\text{series prompt}| - 1000 \rfloor$$

3.5.3 Summariser

With our chunk limit set, we chunk the text and parse the chunk with a prompt to generate our summaries. All significant prompts used can be found in appendices(9.2). We firstly create a general overview summary, which uses only the title and longtitle to summarise overarching details about the legislation. This was done as more of a preview to accompany the title, so that it is more digestible. If we have only one chunk, we can quickly produce a broad and bullet summary. The broad summary is a prompt which simply asks for the entire legislation to be summarised, taking in the whole text. The bullet summary meanwhile asks for a list of the key points from the LLM, which we extract into an array using regex. We then perform a summary on the list of points, and generate our final summary. We perform both to compare the effectiveness of broad summaries against final summaries.

If we have more than a single chunk, we iterate through each chunk, and we prompt it to give a list of the key points (bullet summary). The reason we use a list is that it is a standardised format that all summaries can follow of each chunk. It allows us to work with it more easily, and we can tell the LLM to not repeat points. Similarly, through regex we can extract out the points into an array which gives us greater control. From here we try two different methods to summarise the lists generated from the chunks:

1. **Raw recursive summarisation:** We take the lists, greedily concatenate them together under a new (similarly derived) chunking limit, and prompt an LLM to summarise the list points until they fit into a single summary prompt.
2. **TextRank summarisation:** We take the list of all points, perform TextRank on them, get a list of the most important points, and then greedily concatenate them to fit in a single summary prompt.

Recursive summarisation loops until there is only one chunk, generated under the following condition.

$$\text{new tokens} < 4096 - c - |\text{summary prompt}| - 2048$$

It is important to identify the increased number of tokens we are using, as we want this final summary to be longer and more expansive, which we can help promote using the config and our prompt. We use recursive summarisation as not only has it seen wide usage[156, 136], but with appropriate prompt engineering our LLM could extract the most important points identified by itself. Additionally, for future features, recursive list summarisation lets us have more varied levels of granularity that we can use. However, the drawbacks are even more pronounced with legal texts, as the danger of hallucinations is incredibly high and misinformation is one of the biggest risks we are trying to avoid. To mitigate hallucinations we want the prompts used recursively to be as objective as possible, so we use fewer requirements to make the text easier to understand.

To avoid the risk of hallucinations we use the alternative of TextRank, which will minimise the number of LLM calls we make. Not only reducing the number of possible hallucinations, but also massively increasing speed - especially for larger texts. TextRank has been used in extractive summarisation for years[101], and using it with LLM abstractive summarisation is the next logical step. TextRank is calculated by finding the cosine similarity between all sentences to each other, using any given embedding, akin to STS, and then performing the PageRank[14] algorithm on the resultant matrix. We implement our TextRank as described by Joshi[73], but instead of using GloVe embeddings, we utilize a legal domain BERT embedding[19], which should perform better at identifying similar meanings. We then get a ranked list of the PageRank scores for each point, which should find the most important recurring points that the LLM has determined across all chunks. This comes with the risk that there is not enough of a spread of information between each point, but it is a strong alternative to reduce LLM hallucinations.

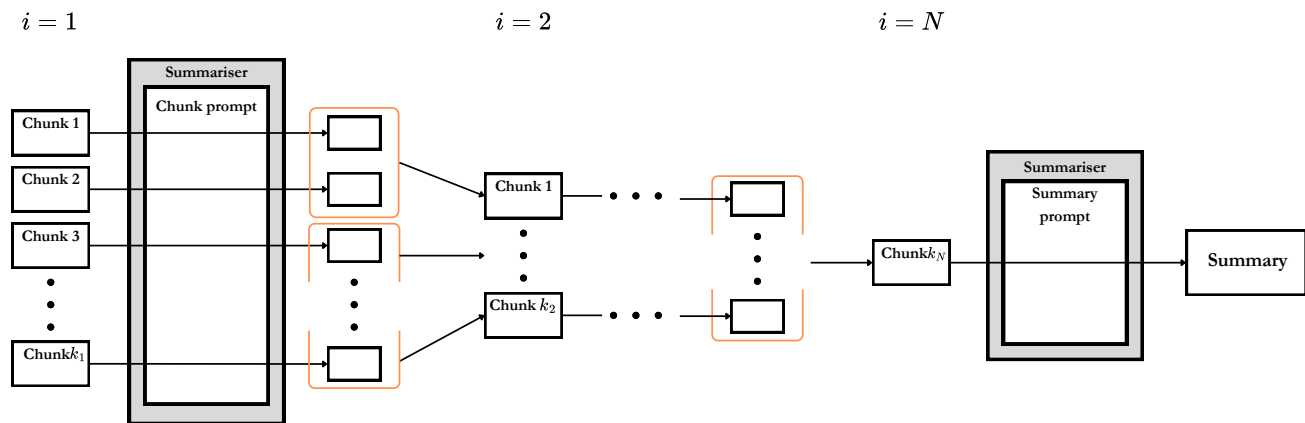


Figure 6: Recursive summarisation visualisation.

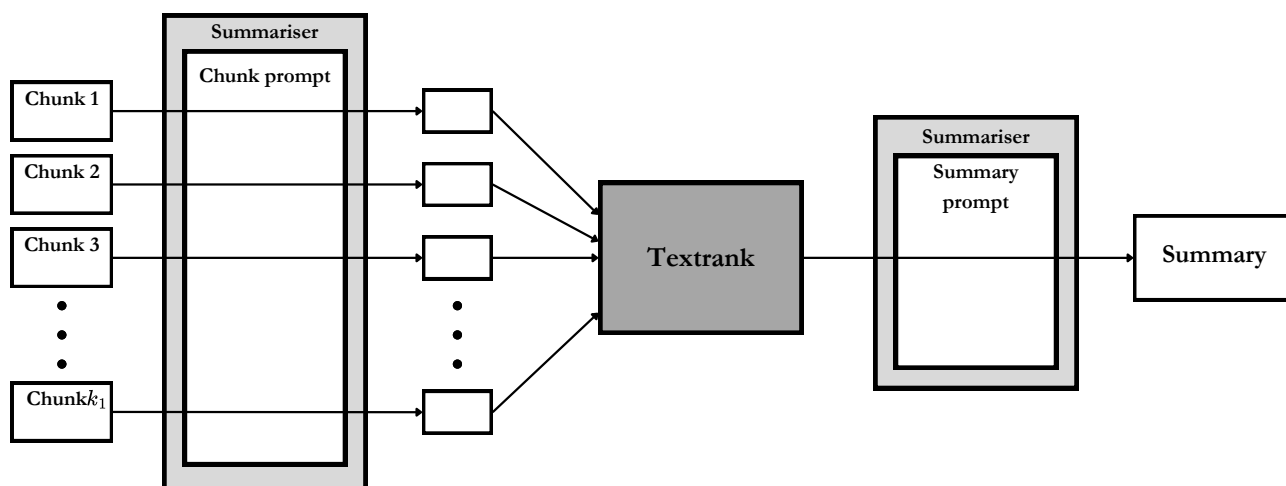


Figure 7: Recursive summarisation visualisation.

The summaries produced fall into three categories.

1. **Short:** The original input is small enough that it does not need to be chunked.
2. **Medium:** The original input after the first bullet summarisation is small enough to summarise.
3. **Large:** The original input after the first bullet summarisation is not small enough to summarise.

Short summaries only use single summarisation, and does not use either recursive or TextRank methods. However medium and large summaries use both so that we can compare which is more successful.

For clarity, here are the all the summaries that are generated:

1. **General overview:** The simple summary of just the title and longtitle.
2. **Broad summary:** If a short result, then we perform a broad summary, which uses the whole chunk as its input.
3. **Chunk summaries:** Creates a list of summary points from a chunk. Every chunk has its respective summary (for every iteration in recursive).
4. **Final summary:** The summary created from a list of key points generated from recursive or textrank.

3.5.4 Prompt engineering

For each part of our summary we produce, we need a respective prompt to our generator. We designed our summariser to take a series of prompts so that we can more easily test and compare how different prompts succeed. There are 5 prompts we need:

1. **general:** This gives the overview of the text from the title and long title.
2. **broad:** This summarises the text when it can all be chunked into a single prompt.
3. **bullet:** This summaries a legislative chunk into a series of bullet points.
4. **bullet to bullet:** This summarises a list of bullet points into a condensed list of bullet points (for recursive summarisation).
5. **summary:** This takes a final list of bullet points of the key clauses, and produces the final summary.

While the specifics between prompts may differ, each prompt will have the same respective structure. Firstly, we need the system message, which was very similar throughout all prompts. A system message is used to specify the role of our LLM, what their goal is and what rules it should follow. We take a lot of inspiration from Boraks prompts[13], with our system message following this form(8).

Our objective depends on the type of summary we are performing. For example in general summaries we specify the objective is to “simplify and provide an overview”. We specify we want to capture the most important impacts and information, which puts the emphasis on what this legislation does, rather than what clauses state. Next we use the system prompt to steer the LLM to generate text that is easier

```
1 <<SYS>> Your objective is <objective> capturing the most important
  impacts and information of the <text>.
2 You are to make your responses as easy to understand and accessible as
  possible, <additional readability suggestions>.
3 Please ensure your responses are unbiased and never share false
  information.
4 If a question does not make any sense, or is not factually coherent then
  do not answer.
5 <</SYS>>
```

Figure 8: Generic system message used by prompts.

to understand than what it is summarising. The additional suggestions were more specific requirements which we experimented with. Then we tell our LLM to be unbiased, so it retains a neutral sentiment; and we tell it to never share false information, so it is less likely to hallucinate. Finally we account for any flawed inputs, where if segments of an input are incorrect - from failed parsing or previous generation - then the LLM will ideally ignore those segments.

After the system message is the given prompt, which gives the respective task that is required. We try to follow the effective prompt engineering principles[62], making our instructions clear and direct. We finally would provide the input, delimited by triple backticks, which was used as common practice in summarisation prompting. The generic instruction format used was developed off of independent experimentation with responses from the LLM, trying to guide and steer it to generating better responses. This process wasn't rigorous, and came from more of a qualitative feeling, as opposed to hard data. This gave us our generic structure(9.2). Once we had this structure in place, we performed tests on 12 different pieces of legislation of varying sizes, using different iterations of the prompt structure, with minor changes to witness the effects(5).

	Prompt	Testing
1	Control prompt	This test was the original results of the prompt we tested against.
2	Vocabulary prompt	This added an additional rule to use 'minimal specialist vocabulary'.
3	Child prompt	This added an additional rule to make the response 'so that a child could understand it'.
4	Explanation prompt	After the [/INST] token we change the OUTPUT to EXPLANATION.
5	Summary prompt	After the [/INST] token we change the OUTPUT to SUMMARY.
6	Contextless bullets	When performing recursive summarisation, we don't include the title and long title.

Table 5: Prompt experimentation.

Prompt	FKGL	SMOG	GF	ARI	CRG	BERTScore
1	8.7443	11.5471	12.2491	10.1318	10.6680	0.7286
2	10.4373	12.9298	14.1765	11.8768	12.3551	0.7330
3	6.5135	9.8406	9.8862	7.085	8.3313	0.7256
4	8.5637	11.4341	12.1867	9.7031	10.4719	0.7246
5	7.8031	10.9125	11.3624	8.5057	9.6459	0.7324
6	7.4477	10.6788	11.1298	8.3519	9.4020	0.7236

Table 6: Prompt experimentation results from 12 differently sized pieces of legislation.

Our analysis of the result sample was to look at their average reading grades, their BERTScores and a qualitative evaluation. CRG is the average of all reading grades across a single legislation, with the idea that the average will encapsulate the complex word interpretations from each metric. From these results we determined that the child prompt was the most optimal as an additional rule, significantly reducing every readability grade, without a major loss in semantic meaning compared to other prompts. Additionally, from reading the summaries we found that they were very similar in content, just the child prompt explained it in clearer words, and shorter sentences. We also found that we wanted to use the summary prompt's ending, as it removed the 'chat' element found in OUTPUT and EXPLANATION. In prompts 1 and 4, summaries would often begin with a 'Hello' or a 'Hey', from a result of the model's training data. In prompt 5, by making it produce tokens after SUMMARY, we essentially trick the LLM into thinking it should immediately start discussing the summary, instead of any surrounding chat fluff. We use a similar trick when generating a list of bullet points, adding a 1. to the end of our prompt. Finally, we decided that we would also implement prompt 6's contextless bullet points. Not only was the average CRG a whole year easier, with no significant difference in BERTScore, but we also found the quality of summaries of the bullet points better. This is likely a result of the title and long title skewing the context of a chunk so that it focuses on less important areas relevant to that section. Therefore, for our final prompt set that we used, we used our generic prompt with the child suggestion, the SUMMARY ending, and the contextless bullets.

3.5.5 Hyperparameter tuning

Due to time constraints, the tuning of the configuration was less rigorous, and primarily consisted of smaller qualitative tests performed on results from our final prompt and other experimental inputs. We defined a generic config to be used in generation when unspecified.

```
config0 = {  
    "max_new_tokens": 1200,  
    "min_new_tokens": 0,  
    "temperature": 0.0001,  
    "do_sample": True,  
    "top_p": 0.95,  
    "num_beams": 1,  
    "repetition_penalty": 1.0,  
    "length_penalty": 1.0,  
}
```

This config was the basis for all subsequent hyperparameters, with only minor changes developing. We decided to set `do_sample` to true, with `top_p` sampling. This was as opposed to greedy sampling, as using greedy sampling often resulted in responses too similar to the original legislation, which were more difficult to read. Sampling gave more creativity to improve the quality of writing. We also decided to set the `temperature` to be very low (≈ 0) to improve coherence. Motivated by Vega’s report[180], we set a high `top_p`=0.95, which resulted in consistently logical and coherent text with richer vocabulary and ideas. We experimented with increasing the `temperature`, and found that readability grade generally increased with respect to `temperature`. Testing wasn’t as extensive as it could have been, hence upon further study we would better fine-tune these hyperparameters.

There were only three configurations that we specialised, which was the general, bullet and summary configs(9.3). The difference between these came in the form of the `max_new_tokens` (discussed in generation(3.5.2)), and the penalties. With a default of 1, `length_penalty` influences how long the text will be. We set the general and bullet length penalties to be -0.1 and 0.9 respectively, aiming to make them shorter and more concise. We set the summary length penalty to be 1.9 as to ideally be more verbose and explain in more detail. The `repetition_penalty` was originally going to be used to try to reduce the number of repeated points generated in a chunk summary, however since the `repetition_penalty` targets repeated use of tokens specifically, it means it is more likely to punish lists. Therefore, we kept it at the default for the bullet config, but increased it slightly to 1.2 for the summary so it would discuss a wider breadth of points. The `diversity_penalty` was not altered, due to a lack of time to experiment with its impacts.

3.6 Metric evaluation

For every metric considered, we wanted to automate each of them so that we could easily analyse the results. Not only is there a large selection of metrics we need to consider, but as we are generating multiple types of summaries, we need to determine how we interpret them. The majority of presented metrics aren’t accurately reflected under addition, meaning if we look at the metrics of a summary in isolation, we may lose our understanding of the broader picture. When displayed on the application, the user should have access to every single summary, even the most granular single points, which means the text we may want to consider would be every piece of text on the page. Therefore, for a given metric we use differing inputs. Before we pass the texts, we parse them into the most appropriate form for the relevant metrics.

3.6.1 Readability

We can calculate all of our desired readability metrics - FRE, FKGL, GF, SMOG, GF - with the text descriptives package[51], which extends the NLTK parser[8] to gather useful details such as average sentence length, average number of sentences, number of words, etc. We investigate the readability of the full original text, every chunk and its respective listed summary, the general overview, the broad summary (if a short text) and the final summary. We also look at three important combinations.

1. Partial summary: the general overview and the final summary.

2. Full summary 0: the general overview, the final summary and the listed chunk summaries.
3. Full summary N : the general overview, the final summary and the N th iteration of recursive listed chunk summaries (if $N > 0$)

These were selected to cover the three key ways that the document may be read, and should represent an approximate upper and lower bound of granularity between the recursive iterations.

3.6.2 Summarisation

These metrics we had to handle differently considering we had an input of two different texts. ROUGE is calculated with the ROUGE scorer library[92], while BERTScore and BLEU is calculated with the evaluate library’s implementations[70]. We used the legal BERT model[19] for our embeddings used by BERTScore. For each metric we compared the same texts: the chunks with each listed chunk summary, and the original full text with the broad summary, partial summary, full summary 0 and full summary N . This lets us check how well information is carried between the key methods of reading, and the original text.

3.6.3 Orthographic and syntactic

These metrics were simpler to calculate, as spelling or grammar mistakes are additive. Therefore, this was performed across both the originally parsed text, and the final full summary. We used LanguageTool[103] which allowed us to customise the list of parsed errors. This included removing any detections of using UK spellings, and detecting grammatical recommendations. The LanguageTool API was not only easy to use and implement, but can also perform automatic corrections, which in future could let us optimise certain inputs or outputs. The metric may also be to detect bad summaries, by having results with errors over a given threshold resulting in being reprocessed or discarded.

3.6.4 Sentiment

Our sentiment analysis was conducted using the prebuilt sentiment pipeline provided by HuggingFace, using the ProsusAI financial sentiment analysis tool FinBERT[3]. This was chosen for several reasons, the most important being ease. At the time of implementation, deadlines were growing more intense and so this was designed with a ‘good enough’ approach. This sentiment analysis tool is designed for financial texts, to determine how positive, negative or neutral the text is. While a legal domain specific tool would be preferable, sentiment analysis of the law is difficult to determine or manually label due to implicit biases[170, 72]. Our goal is to identify how a given text expresses its tone about a given subject, which any sentiment analysis tool would be able to achieve, but due to the very objective nature of legal documents, it often can skew sentiment to be negative. The majority of HuggingFace available sentiment analysis tools are trained on Twitter sentiments¹. When considering how poor public reception is to many legal areas[1, 89], it is easy to see how the topics present in many texts may have been fine-tuned to be considered negative by those models. This is where FinBERT is more appropriate, as not only do many legal texts concern financial matters, but financial sentiment is trained on more objective phrases[95]. FinBERT produces a label of positive, negative, or neutral, which gives us an immediate flag to check. We assume that the original legislation is entirely neutral, and after testing the results of this sentiment analysis tool on several legal texts - both finance and non-finance focused - it found every single text to be strongly neutral, with the score between 0 and 1 being over 0.90 consistently. Therefore, we use this to find the sentiment of the full original text, and the final summary. The reason why we don’t include the listed summaries is that they far more closely correspond to the original text, thus might skew the results to be more neutral when it isn’t.

3.6.5 Information theory

Text descriptives[51] provides the tools necessary to calculate everything surrounding information theory. This calculates entropy and perplexity (which is $e^{-\text{entropy}}$). We use this information as an additional

¹A general trend seen across searched models[61]

measure of text complexity and perform it on the original text and all generated summaries. The intention is to identify if this is a strong measure of coherent responses, and if there are any patterns in outliers we can find.

4 Results

With our pipeline produced, we performed experiments with our methods of summarisation. To test the success of our pipeline, we randomly selected 82 different pieces of legislation between 1995 and 2020, and generated both the recursive and TextRank summaries. We evaluated these pieces of legislation using the metrics described, and qualitative human analysis. If you want to read the summaries generated, they can be found inside of the ‘Legislation Sample’ folder in the code sent with the project.

4.1 Key metrics

The pipeline created summaries successfully, and by looking at every final summary generated, we find that the mean readability grade has massively improved(9). When we compare the UK’s average reading grade of 9 to our scores, we find that both FKGL and ARI were both below the threshold, with GF and SMOG being close to our goal. The average of all grade metrics tells us we are at 9.6855, which is approaching the target threshold such that the British public could understand these texts. Our FRE score significantly improves as well from 39.4147 to 69.4829, going from ‘difficult to read’ to ‘plain english’(15).

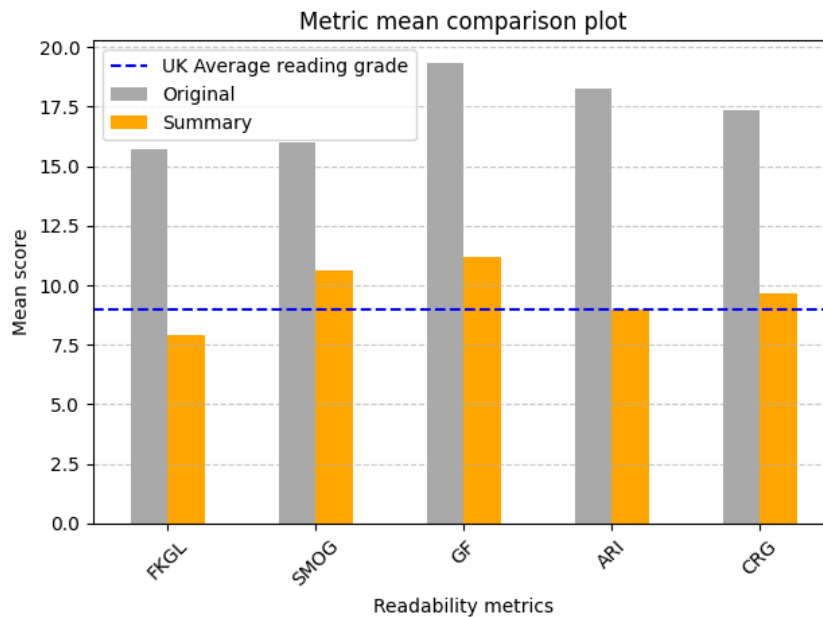


Figure 9: Averages of readability metrics comparing the original to the final summary.

The spread of readability grades across both original and summary texts approximate a normal distribution, following the 68-95-99.7 rule(7), demonstrating that their readability centres around the mean. This tells us that average readability grades are reflective of the majority of the texts, and that both legislation and summaries are consistent in the summaries that they generate. However, this may deviate upon using a larger sample.

	Original	Final	Partial	Full
$\% \in \pm 1\sigma$	70.7317	67.0731	68.2927	67.0731
$\% \in \pm 2\sigma$	92.6829	95.1220	96.3414	96.3414
$\% \in \pm 3\sigma$	100.000	100.000	98.7804	100.000

Table 7: Table of CRG values that lie in different standard deviations of the mean.

If we look at the other groups of summaries, the partial summary performs slightly better in readability, whereas the full summary is noticeably worse. However, we can see that even the full summary with the listed points still is substantially easier to read than the original legal text(8). Both partial and full summaries follow the normal distribution, and generally have higher minimum readability scores, demonstrating that the general and chunk summaries are more consistently the same readability. We can evaluate how effective each type of summary is using this cumulative CRG plot(10), which showcases the percentage of legislation you can read at a given number of years of education. This plot more accurately reflects what our results are, as where our summary curve reaches 1 (max CRG), we can determine that that is the reading age required to understand all pieces of sampled legislation. We also want to consider the steepness of the summary curve, as a steeper slope means our normal distribution is tighter, which will help ensure consistency amongst the readability of our summaries. The total gradient of each slope is approximately equal, which can be understood by their similar minimums and maximums. However, the max gradient of the slope better represents how close all readability scores are together. Looking at max gradient, we find that the full summary has the most consistent readability(9).

	FKGL	SMOG	GF	ARI	CRG	FRE
Original	15.7370	16.0152	19.3100	18.2750	17.3343	39.4147
Final	7.9267	10.6393	11.2052	8.9707	9.6855	69.4828
Partial	7.4719	10.2632	10.7042	8.5408	9.2450	72.5729
Full	10.0373	12.3058	13.4298	10.7812	11.6385	60.7618

Table 8: Average scores of readability metrics across summary types.

$$\begin{aligned} \text{total gradient} &= \frac{1}{\max R - \min R} \\ \text{max gradient} &= \max_{r_i \in R} \frac{\text{cum}(r_{i+1}) - \text{cum}(r_i)}{r_{i+1} - r_i} \\ \text{s.t. } R &= \{ \text{Readability scores} \} \end{aligned}$$

	Max gradient	Total gradient	σ
Final	0.1585	0.0863	2.5152
Partial	0.2439	0.0981	1.6612
Full	0.2805	0.1009	1.1671

Table 9: Measures of CRG spread from cumulative data.

When we look at the summarisation metrics(10), we firstly find that ROUGE and BLEU aren't very useful. Their scores deviate to be very low, despite semantic meaning being retained between texts. This is a result of ROUGE and BLEU not considering paraphrases or synonyms, which are used extensively in our summaries. When we compare ROUGE scores between the partial summary and full summary we see an enormous improvement, which is a result of the listed points acting as more conventional ML summaries, using elements from the original text. The mean of ROUGE-1 across the full summaries suggests they would be 'moderately good'[94], but otherwise ROUGE tells us very little. BLEU is the same, and this was an expected result due to its focus on machine translation. Summarisation incites an implicit loss of textual meaning, derived from the syntax, which is more impactful in BLEU as translation wants to retain the same syntactic meaning as much as possible. BERTScore provides representative results

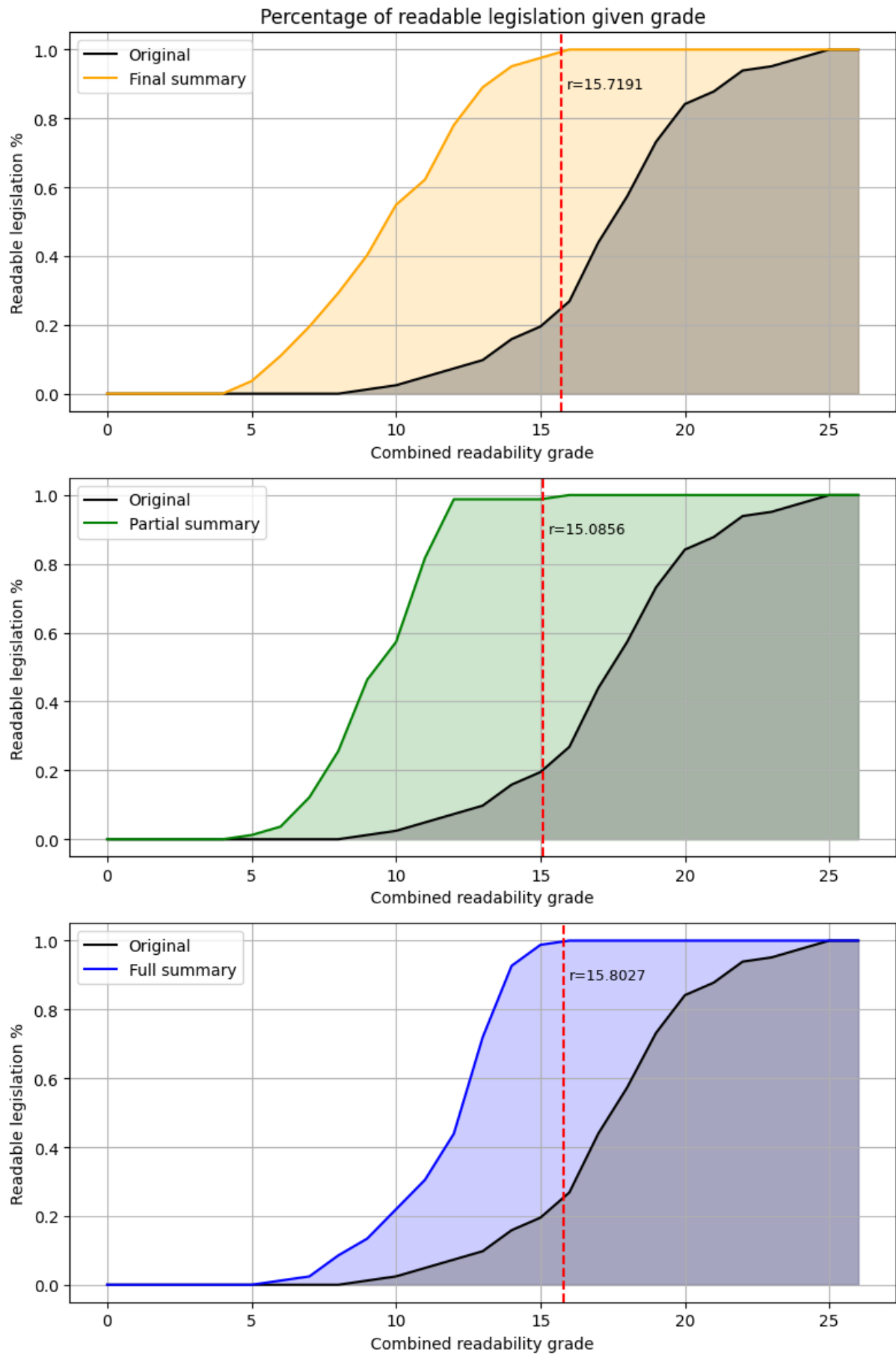


Figure 10: Cumulative CRG readability graphs for the original, partial and full summaries.

with the semantic textual meaning averaging at 0.7356 for partial, and 0.7908 for the full summary. The scores closer to 1 represent closer semantic meaning, supporting that our summaries decently represent the original text. The range of BERTScore is low, with the minimum telling us that the majority of scores will be above 0.7, and should act as a way to determine which summaries are better than others. Additionally, it is important to note that there was no direct relationship between the difficulty of the original text and how well it was summarised.

	Original to partial			Original to summary		
	Mean	Min	Max	Mean	Min	Max
Rouge1	0.1597	0.0082	0.5208	0.4861	0.2323	0.6775
Rouge2	0.0460	0.0038	0.1923	0.2932	0.0574	0.4918
Bleu	0.0139	0.0000	0.1378	0.1042	0.0094	0.3141
Bert	0.7364	0.6941	0.7946	0.7908	0.7181	0.8574
Precision	0.7422	0.6943	0.7926	0.7889	0.7210	0.8623
Recall	0.7310	0.6749	0.7966	0.7932	0.7153	0.8696

Table 10: Summarisation metrics comparison.

Orthographic results of the entire summary unfortunately were not useful, as there were too many suggestion rules that hadn't been accounted for. We looked at the small subset of summaries which had over 20 errors and found that errors that were flagged were often miscellaneous or small spelling differences found between languages, such as hyphenation. Therefore, in their current state, orthographic measures of the summary weren't effective to determine any text failures. However, the average number of errors found in the original parsed text was 731, compared to the average of 9 in the summaries. This indicates our parsing results in multiple spelling or grammatical errors, which the LLM manages to ignore or solve.

Our sentiment analysis result determined that 100% of summaries are given the neutral label. With the assumption that all legal texts are written to be neutral, we demonstrate that all summaries are very close to the original text's sentiment. This can be seen in the sentiment differentials plot(11), where 78.04% of summaries have their sentiment within 0.05 of the original's. However, this isn't necessarily a strong correlation, with a Pearson correlation coefficient $\rho = 0.5561$. We can see from the scatter plot(12) that neutral sentiment groups toward (0.925, 0.925), spreading out from it. If this trend is maintained on larger sample sizes, we can be confident that our summaries are very neutral. Looking at the least neutral texts in the sample, they included heavy subject material with very negative connotations. The 3 least neutral texts covered topics of female genital mutilation, aviation offences and knife crime[122, 119, 128], which each would be treated as negatively by most sentiment models.

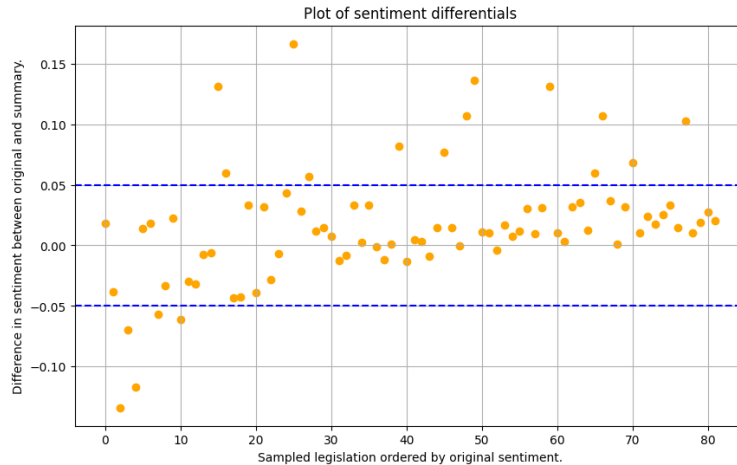


Figure 11: Sentiment differentials graph ordered by original sentiment.

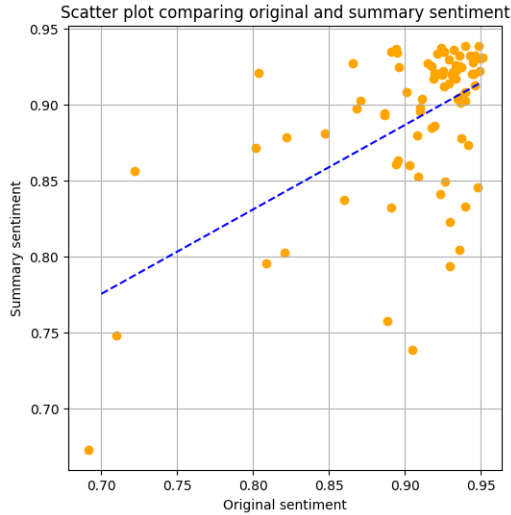


Figure 12: Sentiment scatter plot of original against summary sentiments.

With entropy, we were less certain of what we wanted to identify. Finding the Pearson correlation coefficient between every other metric, we found that entropy has a strong relationship with the number of tokens ($\rho = 0.9997, 0.9488$ for original and summary respectively). There was also a relationship with the original’s orthographic errors. This upon closer inspection seems to be a result of the longer texts having more errors after parsing. Otherwise, entropy did not display any meaningful connections other than identifying longer texts. This may be a result of entropy being a measure of randomness in a text, and in longer texts there is a higher probability of more varied concepts (tokens) appearing. In future studies we will want to explore the potential qualitative relationship the texts have with entropy.

4.2 Recursive and TextRank

TextRank was implemented to minimise hallucinations induced by recursive calls to the LLM. While we were to accept some loss in levels of readability, we actually found the average readability to be near identical(13) with a cosine similarity of 0.9999. Furthermore, we found that their average BERTScore to be very similar at 0.7349 and 0.7304, which means they closely share the same semantic meaning gathered from the original text. The similarity in metrics indicate that both methods are near indistinguishable, hence we performed independent qualitative analysis.

Our process had us take 10 recursive and TextRank summaries of medium or large size, and directly compared their written quality, to identify broader differences. Across all 10, TextRank was generally the better summary, discussing elements of the text in an appropriate amount of detail. Between both, no major hallucinations were detected, only two misleading descriptions present in the recursive summaries. TextRank wasn’t consistently better, occasionally having their responses be very repetitive and focus on a single topic. The text which demonstrated the most discrepancies was 2016 Finance act[124]. This was the largest act sampled at over 100,000 words, and is very difficult to summarise effectively. The recursive summary had the flaw of unequally explained points. With five key points to describe the text, there was an unreasonable amount of detail focused on ‘Fund of Funds’, which was very poorly written, devolving into keywords with no explanation or coherence(14). When looking through the iterations to see what may have occurred to cause this, we found that each subsequent iteration created more ‘nonsense points’, which was either hyper specific detail, or condensed points from previous iterations which were combined inappropriately. This seemed to start to only happen after 4 iterations. On the other hand, TextRank covered ten different broader points in shallow detail, and while it isn’t enough to comprehend the whole legislation, its response is at least consistently coherent. Therefore, since their scores indicate them to be meaningfully similar, upon closer inspection we find that TextRank is the better option. With long texts TextRank avoids this LLM decay that can make elements incoherent and illegible. Furthermore, TextRank is significantly faster to generate, and while we may lose multiple levels of summary, the control we get from TextRank to use other statistical strategies in the future, makes it preferable to use.

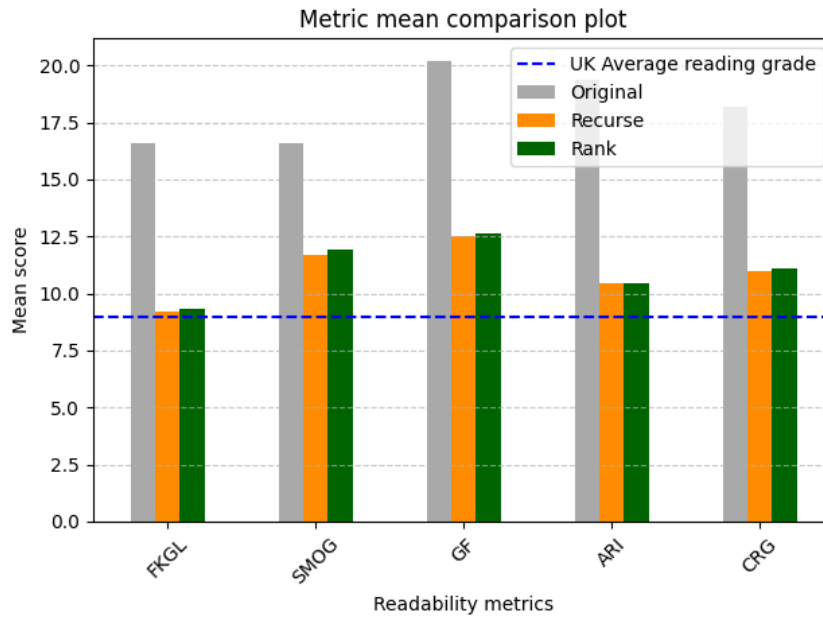


Figure 13: Averages of readability metrics comparing the original to the final summary comparing recursive and TextRank.

```

1 "Fund of Funds: Reduction of carried interest, relevant disposal,
investment life of the scheme, external investors, investment management
services, secondary funds, and large businesses must publish their tax
strategies."

```

Figure 14: Extract from the 2016 Finance act recursive summary.

4.3 Impact of length

There are three different lengths of summary that we can have(3.5.3), which each have an impact on how the summary is generated. Medium sized texts were the rarest in the sample, with there only being 5, however we concluded that medium had no notable differences to large summaries. The differences between short and large texts were significant, where if we compare their final summaries(11) (not the broad summary), we find that the short summaries are significantly easier to read, with an average CRG of 7.52 compared to large texts with an average 11.01. This combined with the subtle difference in average sentiment (from 0.876 to 0.901), we investigated why the summary of the smaller text had such an improved readability score. Sampling 10 short summaries, we found that the majority of them suffered a similar issue of acting like a chatbot. Not only was there greetings in the text, but delivery was very informal, with phrases like “And guess what!” or “Well let me tell ya!”. In addition to the exorbitant usage of exclamation marks, many of the short summaries produced were inappropriate, especially with the subject matters of certain texts. The final summary prompt used was identical to the one used for large texts, hence it is unclear what causes this. However, this isn’t the only summary generated for short texts. Using a broad summary, which has a different prompt, we find our desired results. The broad summaries in each sample was formal and appropriate, retaining the desired information we wanted. The readability scores generated by the broad summaries suggest this as well, and while they may be more difficult to read, they are far more appropriate.

Upon further inspection of short texts, we identify the worst summaries to be the texts with very low readability scores. We looked at the lowest readability scores for TextRank and found that while they weren’t as egregious as short texts, they were a bit too simple to be useful. The lower bound of coherent and appropriate readability scores seemed to be 4, however the range of 8-10 gave the best results. This would need further experimentation with more low readability grade cases.

	FKGL	SMOG	GF	ARI	CRG	FRE
Small (final)	5.8337	8.8760	9.0917	6.4892	7.5727	82.0053
Small (broad)	9.0601	11.4746	12.3595	9.9300	10.7060	64.6220
Large (final)	9.2303	11.7363	12.5097	10.5939	11.0175	61.5756

Table 11: Readability metrics comparison between small and large text summaries.

4.4 Qualitative analysis

While qualitative analysis has been performed throughout the project, the majority of it was conducted independently, with my biased evaluation. Due to the time-consuming nature of reading the summary and the original legal text, as well as needing some level of knowledge to determine if the summary is successful, it made human evaluation difficult. We conducted a small sample of human evaluation of our results with two approaches. We had a detailed evaluation group and a surface level evaluation group. Our detailed evaluation involved two investigators each reading through 10 randomly sampled final summaries and their respective original texts. They would then award scores in different categories: readability, coherence, detail, and hallucinations, each rated on a scale of 1 to 5. We limited the range of the score to force harsher conclusions that would be easier to evaluate. They were allowed to read in any order desired, and could compare and contrast the texts directly. They also gave their overall opinions on the success of the summaries in general. The focus of this study was to primarily identify the appearance of hallucinations. The surface level evaluation group consisted of 12 people that would be shown a summary, give a score of readability, coherence and detail, then be shown the original text, and then rescore using the same categories. This would be performed two times per participant. This study was to act as an evaluation of the quality of summaries compared to the original text without detailed reading. All summaries used were from the broad and TextRank subsets respectively (the optimal summaries from small and large texts).

Participant 1				Participant 2			
R	C	D	H	R	C	D	H
2	4	1	1	3	5	2	1
3	5	2	1.5	3.5	4	2.5	1
3.5	4	2.5	1	4.5	5	2.5	2
4.5	5	2	1	5	4	2.5	1.5
4	4	3	1.5	3	5	3.5	1
3	5	2	1	4	3	2	1
4	3	2	1.5	5	4	3.5	2
5	4	3.5	2	4.5	4	4	2
3	4	2	1	4	4	2.5	1
4	5	2.5	1.5	5	3	3	1.5

Table 12: Detailed evaluation results of 10 randomly sampled pieces of legislation.

	R	C	D
Initial reading	4.125	4.291	3.041
Second reading	4.167	4.291	2.458

Table 13: Mean summary evaluation scores of the 12 participants before and after seeing the original text.

The results from the detailed evaluation(12) found that the readability and coherence of most samples were consistently adequate or good, achieving an average of 3.85 and 4.25 respectively. The detail score was noticeably lower, with an average of 2.475. The lack of detail described was that in longer legal texts, too much information had been abstracted away, so while it was an adequate summary, it had lost

some key tenants of the text. The hallucinations score was to evaluate if the summary had created any fictitious information, and the agreed consensus between both investigators was that this was rarely the case. The few cases they identified were less ‘hallucinations’ and more a lack of detail, that made the text misleading. The common trend seemed to be that the summaries didn’t make up any information, only simplified it down substantially. The investigators were only provided the final summary, whereas the app would provide the individual chunk summaries, therefore this issue of detail may be able to be mitigated in future iterations. The surface level evaluation gave similar results, although they averaged slightly higher(13). The text was considered widely readable and coherent, and this was maintained between seeing the original text. The detail was vastly different, with a drop of 0.583 between seeing the original text. This suggests that many saw the summary to contain more detail than it actually possessed.

The human evaluation may have some issues of population, as the surface level analysis was conducted by members of Warwick University, meaning their education grade is much higher than the average member of the UK populace. Additionally, the detailed evaluation suffered from both investigators not being knowledgeable in the legal domain, meaning there were elements that they couldn’t fully determine if the meaning had been carried between the original and summary text. If there was more time, we would try to perform similar experiments with a more appropriate demographic, to glean more useful results.

5 Project management

With this project taking place under harsh time constraints, with other modules and coursework to be considered, from the outset the requirements were designed to be flexible. The original intention was to develop a complete application, however the modularity of our requirements allowed us to refocus the project upon reevaluation. As we wanted to be flexible, we followed a developer driven Agile approach where we took the role of both stakeholder and developer. Each sprint we would evaluate progress, and change our goals for the following sprint and the remainder of the project.

The project itself was divided into two equal 4 month phases: research and development. We dedicated half of the project to thoroughly learn the topics needed for each of our modules, to account for our lacking initial knowledge. Research performed was dictated by our MoSCoW priorities, and we laid out a Gantt chart to depict our planned progress(15). We followed 2-week long sprints as research required a longer time to accumulate necessary details to determine the trajectory of the project. At the end of our research phase, we determined what was achievable, and created the design plan for the development phase with a similar Gantt chart(16). We followed 1-week long sprints for the development phase, as implementation gave concrete results and objectives to identify. The phases did have overlap of research and implementation between them, but the division served to guide the core goals during a sprint. We built each plan around important deadlines, such as the presentation, as well as external commitments like known coursework deadlines from other modules. The Gantt charts were a projection of what we hoped to achieve, keeping us on target. Due to the expansive nature of the project, we wanted to avoid too much scope creep in one specific module, hence the projected timetables gave us that focus.

While the timetables were designed with flexibility in mind, we greatly overestimated what we could achieve. This was evident by the third sprint, in which we began plans to cut back on scope. We were 5 weeks into research of text simplification and using LLMs, and due to our unfamiliarity with the subject, it resulted in heavy setbacks to our original timetable. Scope was further reduced in subsequent sprints, when the external commitments to other modules increased. At the end of the research phase, we decided to redirect the goals of the project to instead create a simple framework, creating the tools and structures to develop from in future development. The front-end application was where scope reduced the most, with the database, UI and live updating system modules being discarded, to make time to optimise the text simplification component. Due to the flexibility and modularity of our requirements, it let us pivot our project seamlessly. The development phase was more successful with our redefined goals, with more specific targets which made it easier to evaluate success. From our final timeline(17) we see how we diverted quite significantly from our original plan; but we also see how we successfully adapted.

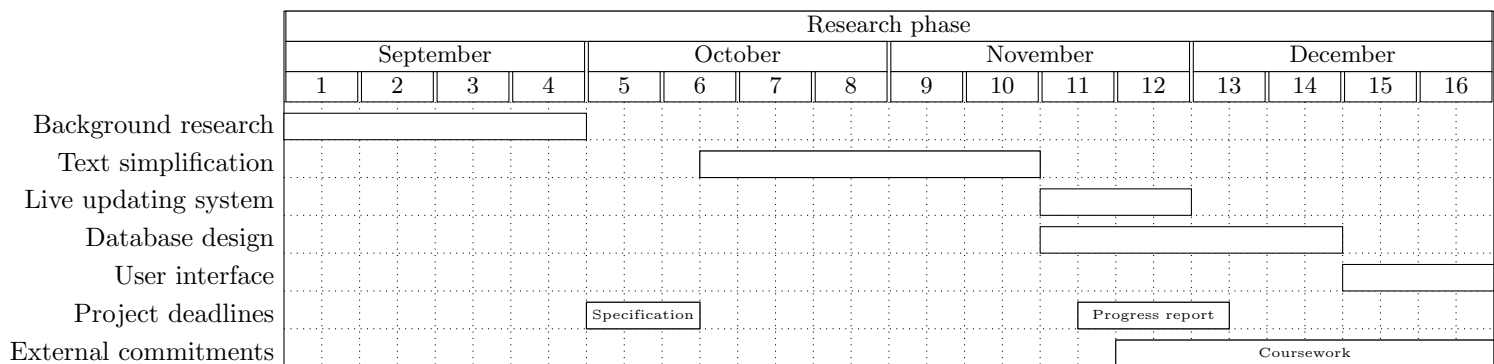


Figure 15: Research phase original Gantt chart.

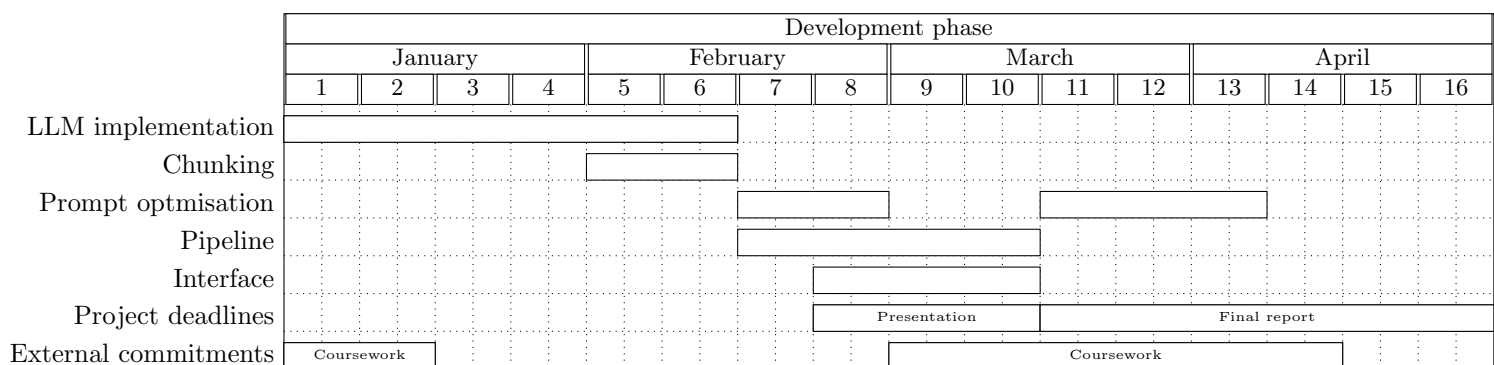


Figure 16: Development phase original Gantt chart.

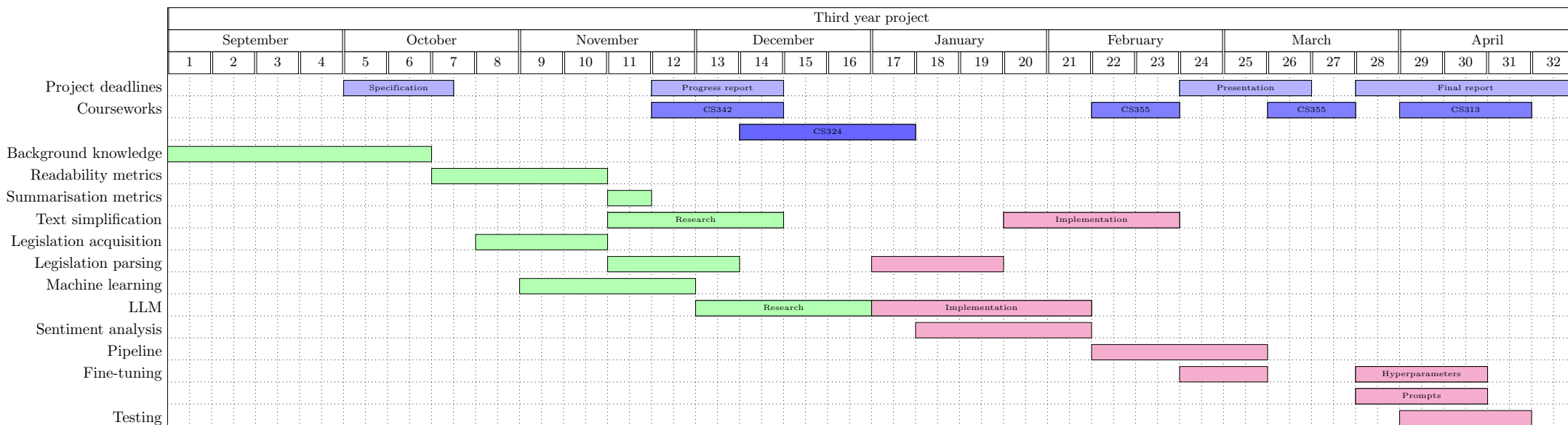


Figure 17: The timeline of development across the entire project.

6 Ethical discussion

Any project surrounding law and politics inherently carries ethical questions. Not only do we concern ourselves with the macro impact of this application, but how it may impact an individual and their life. While we demonstrated in our background that there is a demand for an easier method to read and understand legal texts, we need to consider the public and the individual’s relationship with the law, to evaluate if this project is an appropriate solution to the problem proposed.

The need for this project extends beyond demand, as it could bring about increased trust between the people and the Government. With only 35% of the population stating that they trust the Government[89], it paints a picture of poor political efficacy throughout Britain. When 58% of the populace do not believe they have a say in what the Government does, we can see a nation that is detached from the laws passed by Parliament. Lower literacy levels have been strongly correlated with poor political efficacy[45], which indicates that this divide stems from difficulty to know what the Government is actually doing. As we have discussed, legislation is far from accessible for the average British citizen, and this reflects the lack of perceived transparency. Transparency has been shown to improve political engagement, and cultivate greater accountability for democratic governance[5]. Our finalised application would be an answer to this lack of transparency, and would close the literacy gap that we face. However, just as it could improve transparency, it could be used to further sever the relationship between the Government and the people, through misinformation and manipulation.

Transparency is only as truthful as the information that is distributed to the public. Misinformation or, ‘fake news’ as coined by the media, is a cloak that serves to undermine transparency, planting mistrust in both the Government[111] and journalism[37]. It has become extremely prevalent in media and politics since 2016, and was exacerbated by the global pandemic, with recent studies telling us that 64% of adults say fake news causes confusion on the basic facts of current events and issues[105]. And now misinformation has become a weapon of manipulation wielded by political actors. Oxford found over 93% of surveyed countries uses disinformation as a part of political communication[116], including political parties, governments, and private firms. It is not only widespread, but it has evolved into a business[41]; and exploiting technology and social media is how they profit. Botting has been one method of misinformation that has been used throughout the pandemic[57], and elections throughout the west[79, 11, 81], with the aim to cause disruptions and polarisation. The consequences of this misinformation is that readers become “lost in the of political news”[80]. Facts are obscured and it becomes more difficult for people to tell what is the truth, thus transparency becomes void in the confusing discourse. While our project has demonstrated success in minimal hallucinations, with high BERTScores to suggest the retention of meaning, that doesn’t neglect the impossibility of it spreading misinformation - especially when considering our recursive LLM decay, or inappropriate short summaries. Without absolute guarantees, we run the risk of accidentally distributing misinformation to the public, adding another source of confusion to the truth. There are growing fears of the malicious usage of LLMs and generative AI in the 2024 elections[16], which opens the door to our application being used to intentionally spread false or biased information. We have seen that malicious actors will try to manipulate public opinion[116], Gorton arguing modern technology has begun to trivialise political manipulation[109]. Using our application as their weapon, they could easily alter it to serve their political leanings. By just changing the prompts slightly, we can skew our LLM’s responses to be more critical or supportive of any bills of our choosing. Used in politically motivated hands, this application could become another front to peddle misinformation, or worse, be presented as truthful enough that a portion of the population will interpret it as entirely accurate unbiased information.

Another concern is the risk to the individual. While the main intention behind this application was to keep the UK populace better informed, there is another usage that was not considered. Both the Hodge Jones & Allen report[1] and the Legal Services Board legal needs study[9] showcase that the public has a poor understanding of the legal process. Lower legal confidence often resulted in self-representation, which was associated with lower satisfaction at the outcome. Our project may increase the legal confidence of individuals with legal needs, and may encourage them to self represent. Our shallow and less detailed summaries of the law may mislead people into thinking that they have a greater understanding than they actually do. As our results demonstrated, summaries were not detailed enough compared to the original text. Moreover, our surface level human evaluation group found the summary more detailed from before they had seen the legislation, which indicates that it is possible to wrongly think our summaries

are sufficient replacements for real legal expertise. This runs the ethical risk of our app being used to guide and advise individuals through legal issues, which given our results is not appropriate and may be dangerous to an individual's livelihood. While we can add warnings and recommendations to find real legal aid, it is another risk that has to be considered in our project's usage.

With the potential to negatively impact the individual and the community, either utilised as a conscious source of misinformation, or misused by the user to their detriment, there are many ethical concerns that hold this project back. Generative AI can be used as a powerful tool of social manipulation, and as it enters the realm of politics, we consider the devastation to democracy it may bring. However, technology is not leaving the political or legal domain, and while it may be used maliciously for control, that doesn't stop new approaches being implemented to be used to combat the threat they bring. While our application could be used as a weapon to manipulate, it can also be a beacon to illuminate the truths that misinformation has shrouded. So long as we can implement secure safeguards to avoid misinformation, and guidance to users on how to best use the app, we can build trust and transparency.

7 Evaluation

The original core goal of this project was to find a method to make UK legislation more readable and accessible to the average member of the public, and on the surface, this appears to have been achieved. Our results suggest that our summaries and method to create them are well-founded and could be used to better inform the public. In isolation, our adjusted requirements for our project were successful, but when looked under a harsh ethical lens, our product is far from complete.

Despite the flaws in management and design, our project did achieve the requirements that were decided upon reevaluation. Once the original scope of the project was fine-tuned to more achievable goals, the majority of those targets were reached. Both the metrics and qualitative analysis showcased that the final summaries are useful to gather a surface level knowledge of the legislation, and what its primary effects are. The feedback from our investigators and participants was that they were mostly informative and helpful. With an average readability grade of 10 years of education being just shy of the target 9 years, our summaries alleviate much of the difficulty of reading the legal texts. This represents the massive improvements in text simplification that LLMs provide, and how we can use complementary methods like TextRank to enhance our results. Additionally, our algorithm is effective at covering documents of a range of sizes, with investigations on how our prompts impact small or large texts, something not as explored by other solutions[117].

While we developed a successful framework for analysing acts, one major limitation of our result was being unable to easily parse bills. As discussed (3.2.2), while we programmed methods to access the text of the bills, it was far less consistent and standardised, which made it difficult to parse. This aspect was unfortunately left unfinished, which left out an incredibly important feature. Due to how modular our pipeline is, it does mean in the future we can focus on parsing bills, and it would easily be able to be integrated with the current system. Another limitation of our result was the lack of a complete application. While an interface was created for demonstration purposes, it couldn't be used as an actual tool to query and understand legislation. Most of the reduced scope problems arise from our early project management. The independent Agile methodology, and the ambitious initial scope heavily impacted the project, even if we adapted quickly. However, the greatest limiting factor was our limited background knowledge in ML and LLMs, which made rapid development sometimes difficult. This is demonstrated by the long time it took to get the LLM working on our systems, significantly delaying the project. What exemplified the research difficulties was just how new this technology is. LLMs and their commercial use only became prevalent in the past 2 years, with a large amount of our citations coming from papers that came out during the project itself.

We also consider that many elements of this project adhered to a 'good enough' principle. This practice has led to certain modules having limited success, such as our parser, which generates the structure with some spelling and grammatical errors. This seems to be a result of using `html2text`, combining words and sentences together; as well as not enough text sanitization, removing links and references. Additionally, our readability metrics may not have been as appropriate to use as we thought, with most readability measures being too simplistic to truly encapsulate the complexity of a text. Moreover, the final sample size of 82 pieces of legislation was too limited, due to resource issues at the end of the project. A much

larger sample size would help justify the conclusion we come to. Furthermore, our results themselves may be biased due to Llama2 training data. Meta hasn't revealed what Llama2 was trained on, but it is very plausible that the legal texts we were summarising was a part of that training set, or online papers which discuss those texts. We did not perform extensive experiments on guaranteed unseen texts, which means our algorithm may be significantly worse on newer bills. Our qualitative human evaluation may also be inadequate as our two investigators had no experience with the legal domain, so their evaluations may be missing crucial information. In the future, we would like to use methods similar to Garimella's paper[40] using legal experts.

When we thrust our project under ethical scrutiny, in its current state it is not fit for release. However, we are confident that with the implementation of appropriate safeguards and more extensive testing, that this project can come to fruition. Firstly, far more rigorous qualitative testing needs to be conducted using legal professionals. This may be assisted with the development of better techniques to detect LLM hallucinations, and detecting discrepancies in meaning between two texts. Next we would need harsh conditions using our metrics to rule out invalid responses. This would involve better sentiment and bias tests; as well as hard boundaries on readability metrics to avoid the short summary problem. Additionally, we would ensure that the user understands that the summaries may not always be 100% accurate, and recommend legitimate aid for users with legal needs. These developments are the minimum to justify this project from an ethical viewpoint.

There are many more areas our project could expand upon as well, as there are some aspects to our pipeline that we simply did not have the time to improve.

Further prompt and hyperparameter fine-tuning: We could perform more tests to optimise generation from summaries with more extensive prompt and config testing. This would involve experimenting with more values and inputs to learn patterns in our results. The config couldn't be trained as there is no good function to prevent incoherence, but we could manually find ideal hyperparameters. Prompts could additionally be enhanced with one-shot training[133].

Utilisation of assisted readability tools such as Rewordify: Existing tools like rewordify[148] assist reading complex paragraphs for people with ADHD, highlighting difficult words and explaining their meaning; or rephrasing certain sentences to be more approachable.

Using a pretrained summarisation model such as INKBOT: Instead of Llama2 on its own, we could use a pre-trained Llama2 summarisation model which was trained to summarise information. An example we found was Inkbot[15], which uses its own prompt structure to optimise summaries. We could also use larger models, like Llama2 70b, which has been shown to be more successful.

Improved metrics: Utilise or train better readability metrics which use more features. Additionally investigate other sentiment pipelines, such as Reji's bias detection model[147]

Fine-tuning the Llama2 model to the legal domain: We could write our own example summaries for our model to use, explicitly training it to better summarise complex topics and ideas present in legal texts. We would want to create a training set using the UK parliamentary glossary[125], and summaries created by people more familiar with the legal domain.

Short chunk summaries: Adding more levels of detail, where for each chunk we summarise that specific chunk such that it can be understood in isolation.

Impacted demographic identification: We can prompt the LLM to generate a list of impacted demographics. This list can then be used to notify impacted demographics, or use train of thought generation[train of thought] to elicit how that demographic is affected.

Alternative techniques to TextRank: TextRank finds the top sentences which are most important from every chunk, however we could instead use topic modelling to derive the most important topics that a piece of legislation concerns. This could be performed using K-means clustering[106] or Latent Dirichlet Allocation[77].

Mobile application with live updates: We want to eventually create a mobile application, which would be an accessible method to read our summaries. This would allow for search features, as well as updates of bills that are being passed that may impact user demographics. We would want to lay out teaching mechanisms and explain how someone can do something to affect the passing of a bill[127].

8 Conclusions

Law is complicated, but it doesn't have to be. Throughout this project, we have demonstrated the importance and need for a mechanism to broach the divide between the public and the law, and we have shown it is possible to close this gap. While this project is only in its infancy, it has expressed its enormous potential to be improved. We achieved close to the target average reading age of the UK, and that was with minimal fine-tuning. Going into the project, we were unsure how successfully an LLM would be able to capture the information in the law, but our results revealed that they are the future of NLP tasks. Unguided, LLMs can be unpredictable and unhelpful, but with the correct prompting and hyperparameters, combined with appropriate statistical methods like TextRank, we can generate useful, meaningful outputs. There may still be a lot of work required to ensure this software holds under ethical scrutiny, but we have shown it is possible to simplify the laws, to make them more accessible to the public. The framework is now there, we just need to build upon it, to help restore the UK's trust in our Government.

9 Appendices

9.1 Score contexts

Score	US Grade	UK Grade	Age
< 0		Reception	3-5
0	Kindergarten	Year 1	5-6
1	First grade	Year 2	6-7
2	Second grade	Year 3	7-8
⋮	⋮	⋮	⋮
11	Eleventh grade	Year 12	16-17
12	Twelfth grade	Year 13	17-18
13	College	University	18-19
⋮			⋮
16			21-22
> 16	Graduate	PHD	Adult

Table 14: Approximate relationship between readability grade scores, US grades, UK grades and age.

Score	School Level (US)	Notes
100.00–90.00	5th grade	Very easy to read. Easily understood by an average 11-year-old student.
90.0–80.0	6th grade	Easy to read. Conversational English for consumers.
80.0–70.0	7th grade	Fairly easy to read.
70.0–60.0	8th & 9th grade	Plain English. Easily understood by 13- to 15-year-old students.
60.0–50.0	10th to 12th grade	Fairly difficult to read.
50.0–30.0	College	Difficult to read.
30.0–10.0	College graduate	Very difficult to read. Best understood by university graduates.
10.0–0.0	Professional	Extremely difficult to read. Best understood by university graduates.

Table 15: Flesch reading ease score interpretations[38].

9.2 Prompts

Generic prompt selection devised for experimentation.

General prompt:

```
1 <s>[INST] <<SYS>>
2 Your objective is to simplify and provide an overview of a given piece
3 of UK legislation, capturing the most important impacts and information
4 of the legislation.
5 You are to make your responses as easy to understand and accessible as
6 possible.
7 Please ensure your responses are unbiased and never share false
8 information.
9 If a question does not make any sense, or is not factually coherent then
10 do not answer.
11 <</SYS>>
12 Given is the title and long title of a piece of UK legislation. Explain
13 what this legislation does in very simple terms. Make it very easy to
14 understand what it is doing.
15 ```TITLE: {TITLE}
16 LONG TITLE: {LONGTITLE}```
17 [/INST] OUTPUT:
```

Broad prompt:

```
1 <s>[INST] <<SYS>>
2 Your objective is to explain and simplify UK legislation, capturing the
3 most important impacts and information of the text.
4 You are to make your responses as easy to understand and accessible as
5 possible.
6 Please ensure your responses are unbiased and never share false
7 information.
8 If a question does not make any sense, or is not factually coherent then
9 do not answer.
10 <</SYS>>
11 Explain and simplify the UK legislation delimited in the triple brackets
12 . Simplify what this act does, and explain the effects of this act,
13 making it easier to understand.
14 Focus on what this legislation does, and who it impacts. Make it very
15 easy to understand.
16 ```TITLE: {TITLE}
17 LONG TITLE: {LONGTITLE}
18 CLAUSES: {BODY}```
19 [/INST] OUTPUT:
```

Bullet prompt: (single)

```
1 <s>[INST] <<SYS>>
2 Your objective is to explain and simplify UK legislation, capturing the
3 most important impacts and information of the text.
4 You are to make your responses as easy to understand and accessible as
5 possible.
6 Please ensure your responses are unbiased and never share false
7 information.
8 If a question does not make any sense, or is not factually coherent then
9 do not answer.
10 Your answer must be in numeric bullet points.
11 <</SYS>>
12 Explain and simplify the UK legislation delimited in the triple brackets
13 . Simplify what this act does, and explain the effects of this act,
14 making it easier to understand.
```

```
9 Focus on what this legislation does, and who it impacts. Make it very
10 easy to understand.
11 Give your answer in only numeric bullet points.
12 ```TITLE: {TITLE}
13 LONG TITLE: {LONGTITLE}
14 CLAUSES: {BODY}``` [/INST]
15 NUMERIC BULLET POINTS:
16 1.
```

Bullet prompt: (chunked)

```
1 <s>[INST] <<SYS>>
2 Your objective is to explain and simplify chunks of UK legislation,
3 capturing the most important impacts and information of the text.
4 You are to make your responses as easy to understand and accessible as
5 possible, using minimal specialist vocabulary.
6 The text you will be seeing will only be a part of the wider text.
7 Please ensure your responses are unbiased and never share false
8 information.
9 If a question does not make any sense, or is not factually coherent then
10 do not answer.
11 <</SYS>>
12 Simplify and summarise with numeric bullet points, this section of
13 clauses of the legislation delimited by the triple backticks. Don't
14 repeat ideas in the bullet points. These clauses are not the whole
15 legislation, only making up part of it.
16 ```TITLE: {TITLE}
17 LONG TITLE: {LONGTITLE}
18 CLAUSES: {BODY}``` [/INST]
19 NUMERIC BULLET POINTS:
20 1.
```

Bullet to bullet prompt:

```
1 <s>[INST] <<SYS>>
2 Your objective is to combine the most important parts of summaries of UK
3 legislation. The text you will be seeing will be all summaries of
4 different parts of one legislation.
5 Please ensure your responses are unbiased and never share false
6 information.
7 If a question does not make any sense, or is not factually coherent then
8 do not answer.
9 Always try to make your lists shorter and take only the most important
10 and relevant points to a casual reader.
11 <</SYS>>
12 Simplify and summarise the list of bullet points into a more condense
13 list of bullet points. The bullet points are each a summary of different
14 parts of a piece of UK legislation. Combine and simplify points
15 together to make a shorter list, while losing as little information as
16 possible. Give your answer as a list of bullet points.
17 ```TITLE: {TITLE}
18 LONG TITLE: {LONGTITLE}
19 BULLET POINTS:
20 {BODY}``` [/INST]
21 CONDENSED BULLET POINTS:
```

Summary prompt:

```
1 <s>[INST] <<SYS>>
```

```
2 You are an AI which takes an input of a list of simplified points from a
   piece of UK Legislation, and makes it easier for readers to understand
   what the legislation does.
3 You are to make your responses as easy to understand and accessible as
   possible, using minimal specialist vocabulary.
4 You will be getting a list of bullet points explaining the key points of
   a legislation.
5 Do not repeat information in your summary.
6 If a question does not make any sense, or is not factually coherent then
   do not answer.
7 <</SYS>>
8 You are to explain what this piece of UK Legislation does in simple and
   easy to understand terms. Make your answer easy to read, using simpler
   words and phrases. Explain any terms that people may not understand.
9 Your output should be descriptive but easy for anybody to understand.
   Try to explain the meaning of what this legislation actually does.
10 Focus your summary on the impacts of the legislation, and how it affects
    people.
11 INPUT: ``TITLE: {TITLE}
12 LONG TITLE: {LONGTITLE}
13 BULLET POINTS:
14 {BODY}```
15 [/INST]
16 OUTPUT:
```

9.3 Configs

```
bullet_config = {
    "max_new_tokens": 1000,
    "min_new_tokens": 0,
    "temperature": 0.0001,
    "do_sample": True,
    "top_p": 0.95,
    "num_beams": 1,
    "repetition_penalty": 1.0,
    "length_penalty": 0.9,
},

summary_config = {
    "max_new_tokens": 2048,
    "min_new_tokens": 0,
    "temperature": 0.0001,
    "do_sample": True,
    "top_p": 0.95,
    "num_beams": 1,
    "repetition_penalty": 1.2,
    "length_penalty": 1.9,
},

general_config = {
    "max_new_tokens": 1024,
    "min_new_tokens": 0,
    "temperature": 0.0001,
    "do_sample": True,
    "top_p": 0.97,
    "num_beams": 1,
    "repetition_penalty": 1.1,
    "length_penalty": -0.1,
}
```

References

- [1] Hodge Jones & Allen. *Innovation in Law Report 2015 UK Perceptions of the Legal and Justice System 4*. Dec. 2015. URL: https://www.hja.net/wp-content/uploads/HJA_UNJUST_KINGDOM.pdf.
- [2] Shyamal Anadkat and Simon Fishman. *How to evaluate a summarization task — OpenAI Cookbook*. cookbook.openai.com, Aug. 2016. URL: https://cookbook.openai.com/examples/evaluation/how_to_eval_abstractive_summarization.
- [3] Dogu Araci. *FinBERT: Financial Sentiment Analysis with Pre-trained Language Models FinBERT: Financial Sentiment Analysis with Pre-trained Language Models*. 2019. URL: <https://arxiv.org/pdf/1908.10063.pdf>.
- [4] AskYourPDF. *How to Use ChatGPT to Create a Research Paper Summary*. AskYourPDF, Oct. 2023. URL: <https://askyourpdf.com/blog/how-to-use-chatgpt-to-create-a-research-paper-summary>.
- [5] World Bank. *Evidence on the Impact of Transparency*. June 2016. URL: <https://thedocs.worldbank.org/en/doc/492041464199876576-0050022016/original/Chapter6Evidenceontheimpactoftransparency.pdf>.
- [6] Haddon-Cave Barnard; "ENGLISH LAW AND DESCENT INTO COMPLEXITY" GRAY'S INN READING. 2021. URL: <https://www.judiciary.uk/wp-content/uploads/2021/07/ENGLISH-LAW-AND-DESCENT-INTO-COMPLEXITY-1.pdf>.
- [7] Mike Bell and Christopher Badley. *Equi-law*. Equi-law UK. URL: <https://equi-law.uk/>.
- [8] Steven Bird. *nlTK: Natural Language Toolkit*. PyPI, Jan. 2023. URL: <https://pypi.org/project/nltk/>.
- [9] Legal Services Board. *Largest ever legal needs survey in England and Wales*. The Legal Services Board, Jan. 2020. URL: <https://legalservicesboard.org.uk/news/largest-ever-legal-needs-survey-in-england-and-wales>.
- [10] Rowan Bohde. *Document Summarization using TextRank : blog : Rowan Bohde*. rowan.computer, Sept. 2012. URL: <https://rowan.computer/blog/document-summarization/>.
- [11] Noémi Bontridder and Yves Pouillet. "The role of artificial intelligence in disinformation". In: *Data & Policy* 3 (2021). DOI: [10.1017/dap.2021.20](https://doi.org/10.1017/dap.2021.20).
- [12] Reza Bonyadi. *Pitfalls in Semantic Search*. LinkedIn, Sept. 2023. URL: <https://www.linkedin.com/pulse/pitfalls-semantic-search-reza-bonyadi/>.
- [13] Eliran Boraks. *Llama 2 Prompt Engineering — Extracting Information From Articles Examples*. Medium, Nov. 2023. URL: <https://medium.com/@eboraks/llama-2-prompt-engineering-extracting-information-from-articles-examples-45158ff9bd23>.
- [14] Sergey Brin and Lawrence Page. "The anatomy of a large-scale hypertextual Web search engine". In: *Computer Networks and ISDN Systems* 30 (Apr. 1998), pp. 107–117. DOI: [10.1016/S0169-7552\(98\)00110-X](https://doi.org/10.1016/S0169-7552(98)00110-X).
- [15] Adam Brusselback. *TheBloke/Inkbot-13B-8k-0.2-GGUF Hugging Face*. huggingface.co, Sept. 2023. URL: <https://huggingface.co/TheBloke/Inkbot-13B-8k-0.2-GGUF>.
- [16] Seb Butcher. *2024 may be the year online disinformation finally gets the better of us*. POLITICO, Mar. 2024. URL: <https://www.politico.eu/article/eu-elections-online-disinformation-politics/>.
- [17] Vincent Caruana. *Semantic textual similarity (STS) and search*. Algolia, May 2023. URL: <https://www.algolia.com/blog/product/semantic-textual-similarity-a-game-changer-for-search-results-and-recommendations/>.
- [18] Mert Cemri, Tolga Çukur, and Aykut Koç. *Unsupervised Simplification of Legal Texts*. 2022. arXiv: [2209.00557](https://arxiv.org/abs/2209.00557) [cs.CL].
- [19] Ilias Chalkidis et al. "LEGAL-BERT: The Muppets straight out of Law School". In: *Findings of the Association for Computational Linguistics: EMNLP 2020* (2020). DOI: [10.18653/v1/2020.findings-emnlp.261](https://doi.org/10.18653/v1/2020.findings-emnlp.261).
- [20] Jeanne Sternlicht Chall and Edgar Dale. *Readability Revisited: The New Dale-Chall Readability Formula*. Brookline Books, 1995. URL: https://books.google.co.uk/books?id=2nbuAAAAMAAJ&redir_esc=y.

- [21] Peter Clark et al. *Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge*. arXiv.org, Mar. 2018. URL: <https://arxiv.org/abs/1803.05457>.
- [22] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. First Edition. Addison-Wesley, Sept. 1994. URL: <https://www.amazon.com/exec/obidos/ASIN/020162432X/acmorg-20>.
- [23] Meri Coleman and Ta Lin Liau. “A computer readability formula designed for machine scoring.” In: *Journal of Applied Psychology* 60 (1975), pp. 283–284. URL: <https://api.semanticscholar.org/CorpusID:144250124>.
- [24] Edgar Dale and Jeanne S. Chall. “A Formula for Predicting Readability: Instructions”. In: *Educational Research Bulletin* 27 (1948), pp. 37–54. URL: <https://www.jstor.org/stable/1473669#:~:text=the%20number%20of%20words%20in%20the%20sample%20by>.
- [25] IBM Data and AI Team. *Shedding light on AI bias with real world examples*. IBM Blog, Oct. 2023. URL: <https://www.ibm.com/blog/shedding-light-on-ai-bias-with-real-world-examples/>.
- [26] George Denison. *4 shocking AI bias examples*. Prolific, Oct. 2023. URL: <https://www.prolific.com/resources/shocking-ai-bias>.
- [27] Franck Dernoncourt. *Interpreting ROUGE scores*. Cross Validated, Nov. 2017. URL: <https://stats.stackexchange.com/questions/301626/interpreting-rouge-scores>.
- [28] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv.org, Oct. 2018. URL: <https://arxiv.org/abs/1810.04805>.
- [29] Victor Dibia. *Practical Steps to Reduce Hallucination and Improve Performance of Systems Built with Large Language Models*. newsletter.victordibia.com, Feb. 2023. URL: <https://newsletter.victordibia.com/p/practical-steps-to-reduce-hallucination/#%C2%A7why-hallucination-is-here-to-stay>.
- [30] University of Edinburgh. *Linguistics 1A Morphology 2 Complex words*. URL: <http://www.lel.ed.ac.uk/~packema/teaching/Ling%201A%202010-2011%20morphology/Ling%201A%202010-2011%20morphology%202.pdf>.
- [31] Illinois State Board of Education. *Determining Text Complexity Appendix A, p. 4-8*. URL: <https://www.isbe.net/Documents/5-determining-text-complexity.pdf>.
- [32] Mihal Emberton. “Unconscious Bias Is a Human Condition”. In: *The Permanente Journal* 25 (May 2021). DOI: [10.7812/TPP/20.199](https://doi.org/10.7812/TPP/20.199). URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8784036/>.
- [33] Gustavo Espíndola. *Chunk Division and Overlap: Understanding the Process*. Medium, Sept. 2023. URL: <https://gustavo-espindola.medium.com/chunk-division-and-overlap-understanding-the-process-ade7eae1b2bd#:~:text=This%20overlap%20allows%20for%20some>.
- [34] Hans Eysenck. *politics*. www.ditext.com, 1956. URL: <https://www.ditext.com/eysenck/politics.html>.
- [35] Fabiano Falcão. *Metrics for evaluating summarization of texts performed by Transformers: how to evaluate the...* Medium, Apr. 2023. URL: <https://fabianofalcao.medium.com/metrics-for-evaluating-summarization-of-texts-performed-by-transformers-how-to-evaluate-the-b3ce68a309c3>.
- [36] PR Fitzsimmons et al. “A readability assessment of online Parkinson’s disease information”. In: *The Journal of the Royal College of Physicians of Edinburgh* 40 (Dec. 2010), pp. 292–296. DOI: [10.4997/jrcpe.2010.401](https://doi.org/10.4997/jrcpe.2010.401). (Visited on 07/29/2021).
- [37] Jude Fleming. *LibGuides: Fake News: Consequences of fake news*. libguides.exeter.ac.uk, Sept. 2021. URL: <https://libguides.exeter.ac.uk/fakenews/consequences>.
- [38] Rudolf Flesch. *HOW TO WRITE PLAIN ENGLISH*. Nyu.edu, 1981. URL: <https://pages.stern.nyu.edu/~wstarbuc/Writing/Flesch.htm>.
- [39] Kelsey Foster. *Text Summarization for NLP: 5 Best APIs, AI Models, and AI Summarizers in 2023*. News, Tutorials, AI Research, May 2022. URL: <https://www.assemblyai.com/blog/text-summarization-nlp-5-best-apis/>.
- [40] Aparna Garimella et al. “Text Simplification for Legal Domain: Insights and Challenges”. In: *Proceedings of the Natural Legal Language Processing Workshop 2022*. Ed. by Nikolaos Aletras et al. Abu Dhabi, United Arab Emirates (Hybrid): Association for Computational Linguistics, Dec. 2022, pp. 296–304. DOI: [10.18653/v1/2022.nllp-1.28](https://doi.org/10.18653/v1/2022.nllp-1.28). URL: <https://aclanthology.org/2022.nllp-1.28>.

- [41] Joanna Glasner. *It's No Lie: Startups Fighting Disinformation Are Raking In Cash*. Crunchbase News, Feb. 2023. URL: <https://news.crunchbase.com/ai-robotics/venture-startups-disinformation-primer-vinesight/>.
- [42] Google. *Evaluating models — AutoML Translation Documentation*. Google Cloud. URL: <https://cloud.google.com/translate/automl/docs/evaluate>.
- [43] UK Government. *Discrimination: Your Rights*. gov.uk, 2023. URL: <https://www.gov.uk/discrimination-your-rights>.
- [44] Welsh Government. *The layout of an Act — Law Wales*. law.gov.wales, Mar. 2021. URL: <https://law.gov.wales/layout-act>.
- [45] Anke Grotl schen. “The narrative of ‘equality of chances’ as an approach to interpreting PIAAC results on perceived political efficacy, social trust and volunteering and the quest for political literacy”. In: *European Journal for Research on the Education and Learning of Adults* 8 (May 2017), p. 8. DOI: [10.3384/rela.2000-7426.rela0211](https://doi.org/10.3384/rela.2000-7426.rela0211).
- [46] Robert Gunning. *The Technique of Clear Writing*. McGraw-Hill, 1971. URL: https://books.google.co.uk/books/about/The_Technique_of_Clear_Writing.html?id=YqMRtAEACAAJ%5C&redir_esc=y.
- [47] Einar H. Dyvik. *Countries with highest share of population with higher education*. Statista, Oct. 2023. URL: <https://www.statista.com/statistics/1346188/countries-world-highest-share-bachelors-degree/>.
- [48] hackerllama. *Get Llama 2 Prompt Format Right*. Reddit, July 2023. URL: https://www.reddit.com/r/LocalLLaMA/comments/155po2p/get_llama_2_prompt_format_right/.
- [49] Gillian Hadfield. “Higher Demand, Lower Supply? A Comparative Assessment of the Legal Resource Landscape for Ordinary Americans”. In: *Fordham Urban Law Journal* 37 (Jan. 2010), p. 37. URL: <https://ir.lawnet.fordham.edu/ulj/vol37/iss1/6/>.
- [50] Michael Hanna and Ond rej Bojar. *A Fine-Grained Analysis of BERTScore*. 2021. URL: <https://aclanthology.org/2021.wmt-1.59.pdf>.
- [51] Lasse Hansen. *textdescriptives: A library for calculating a variety of features from text using spaCy*. PyPI, Dec. 2022. URL: <https://pypi.org/project/textdescriptives/1.1.1/>.
- [52] G Harry and M Laughlin. *SMOG grading –a new readability formula*. 1969. URL: https://ogg.osu.edu/media/documents/health_lit/WRRSMOG_Readability_Formula_G._Harry_McLaughlin_1969.pdf.
- [53] Kartik Ashok Hawelikar. “Exploring the Impact of Prompt Engineering on ChatGPT 3.5 Text Summarization: A BERT Score Evaluation”. In: *International Research Journal of Modernization in Engineering Technology and Science* (Oct. 2023). DOI: [10.56726/irjmets45268](https://doi.org/10.56726/irjmets45268).
- [54] Agency for Healthcare Research and Quality. *Tip 6. Be Cautious About Using Readability Formulas*. Ahrq.gov, 2015. URL: <https://www.ahrq.gov/talkingquality/resources/writing/tip6.html>.
- [55] Amy S. Hedman. “Using the SMOG Formula to Revise a Health-Related Document”. In: *American Journal of Health Education* 39.1 (2008), pp. 61–64. DOI: [10.1080/19325037.2008.10599016](https://doi.org/10.1080/19325037.2008.10599016). eprint: <https://doi.org/10.1080/19325037.2008.10599016>. URL: <https://doi.org/10.1080/19325037.2008.10599016>.
- [56] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding”. In: *arXiv (Cornell University)* (Sept. 2020). DOI: [10.48550/arxiv.2009.03300](https://doi.org/10.48550/arxiv.2009.03300).
- [57] McKenzie Himelein-Wachowiak et al. “Bots and misinformation spread on social media: A mixed scoping review with implications for COVID-19 (Preprint)”. In: *Journal of Medical Internet Research* 23 (Jan. 2021). DOI: [10.2196/26933](https://doi.org/10.2196/26933). URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8139392/>.
- [58] Tsutomu Hirao et al. *Extracting Important Sentences with Support Vector Machines*. Jan. 2002. URL: <https://aclanthology.org/C02-1053.pdf>.
- [59] Allen H. Huang, Hui Wang, and Yi Yang. “FinBERT: A Large Language Model for Extracting Information from Financial Text*”. In: *Contemporary Accounting Research* 40.2 (2023), pp. 806–841. DOI: <https://doi.org/10.1111/1911-3846.12832>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/1911-3846.12832>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/1911-3846.12832>.

- [60] HuggingFace. *Generation*. huggingface.co. URL: https://huggingface.co/docs/transformers/en/main_classes/text-generation.
- [61] HuggingFace. *HuggingFace sentiment analysis model search*. HuggingFace, Apr. 2024. URL: https://huggingface.co/models?pipeline_tag=text-classification&language=en&sort=trending&search=sentiment+.
- [62] HuggingFace. *LLM prompting guide*. huggingface.co. URL: <https://huggingface.co/docs/transformers/main/en/tasks/prompting>.
- [63] HuggingFace. *Quantization*. huggingface.co. URL: <https://huggingface.co/docs/transformers/quantization>.
- [64] HuggingFace. *Summarization*. huggingface.co. URL: <https://huggingface.co/docs/transformers/en/tasks/summarization>.
- [65] HuggingFace. *What is Sentence Similarity? - Hugging Face*. huggingface.co. URL: <https://huggingface.co/tasks/sentence-similarity>.
- [66] Jason Huggins. *selenium: Python bindings for Selenium*. PyPI, 2004. URL: <https://pypi.org/project/selenium/>.
- [67] Hyperskill. *Overview of text summarization — Text summarization — Main NLP tasks — NLP — Data science — Computer science*. Hyperskill. URL: <https://hyperskill.org/learn/step/25430>.
- [68] Hyperskill. *ROUGE — Classic NLP metrics — NLP metrics — NLP — Data science — Computer science*. Hyperskill. URL: <https://hyperskill.org/learn/step/29669>.
- [69] Iguazio. *What is LLM Temperature*. Iguazio. URL: <https://www.iguazio.com/glossary/llm-temperature/#:~:text=LLM%20temperature%20is%20a%20parameter>.
- [70] HuggingFace Inc. *evaluate: HuggingFace community-driven open-source library of evaluation*. PyPI, Oct. 2023. URL: <https://pypi.org/project/evaluate/>.
- [71] Pranay Jindal and JoyC MacDermid. “Assessing reading levels of health information: uses and limitations of flesch formula”. In: *Education for Health* 30 (2017), p. 84. DOI: [10.4103/1357-6283.210517](https://doi.org/10.4103/1357-6283.210517). (Visited on 12/18/2019).
- [72] Tanya Johnson. *Research Guides: Implicit Bias in the Law: Generally*. libguides.law.uconn.edu, Feb. 2021. URL: <https://libguides.law.uconn.edu/implicit>.
- [73] Prateek Joshi. *Introduction to Text Summarization using the TextRank Algorithm*. Analytics Vidhya, Nov. 2018. URL: <https://www.analyticsvidhya.com/blog/2018/11/introduction-text-summarization-textrank-python/>.
- [74] Josh Kaufman. *About This Repo*. GitHub, May 2022. URL: <https://github.com/first20hours/google-10000-english>.
- [75] Suleman Kazi and Adel Elmahdy. *Top Large Language Models (LLMs): GPT-4, LLaMA 2, Mistral 7B, ChatGPT, and More*. Vectara, Oct. 2023. URL: [https://vectara.com/blog/top-large-language-models-llms-gpt-4-llama-gato-bloom-and-when-to-choose-one-over-the-other/#:~:text=Top%20Large%20Language%20Models%20\(LLMs](https://vectara.com/blog/top-large-language-models-llms-gpt-4-llama-gato-bloom-and-when-to-choose-one-over-the-other/#:~:text=Top%20Large%20Language%20Models%20(LLMs).
- [76] Atif Khan. “A Review on Abstractive Summarization Methods”. In: *Journal of Theoretical and Applied Information Technology* 59 (Jan. 2014), pp. 64–72.
- [77] Kevin Kibe. *Topic Modeling Using Latent Dirichlet Allocation (LDA)*. Analytics Vidhya, Feb. 2023. URL: [https://www.analyticsvidhya.com/blog/2023/02/topic-modeling-using-latent-dirichlet-allocation-lda/#:~:text=In%20NLP\(Natural%20Language%20Processing](https://www.analyticsvidhya.com/blog/2023/02/topic-modeling-using-latent-dirichlet-allocation-lda/#:~:text=In%20NLP(Natural%20Language%20Processing).
- [78] J. Kincaid et al. “Derivation Of New Readability Formulas (Automated Readability Index, Fog Count And Flesch Reading Ease Formula) For Navy Enlisted Personnel”. In: *Institute for Simulation and Training* (Jan. 1975). URL: <https://stars.library.ucf.edu/istlibrary/56/>.
- [79] David Klepper and Ali Swenson. *AI-generated disinformation poses threat of misleading voters in 2024 election*. PBS NewsHour, May 2023. URL: <https://www.pbs.org/newshour/politics/ai-generated-disinformation-poses-threat-of-misleading-voters-in-2024-election>.
- [80] Sarah Kreps. *The Role of Technology in Online Misinformation*. June 2020. URL: <https://www.brookings.edu/wp-content/uploads/2020/06/The-role-of-technology-in-online-misinformation.pdf>.

- [81] Sarah Kreps, R. Miles McCain, and Miles Brundage. “All the News That’s Fit to Fabricate: AI-Generated Text as a Tool of Media Misinformation”. In: *Journal of Experimental Political Science* 9.1 (2022), pp. 104–117. DOI: [10.1017/XPS.2020.37](https://doi.org/10.1017/XPS.2020.37).
- [82] Labelbox. *Zero-Shot Learning vs. Few-Shot Learning vs. Fine-Tuning: A technical walkthrough using OpenAI’s APIs & models*. labelbox-guides.ghost.io. URL: <https://labelbox.com/guides/zero-shot-learning-few-shot-learning-fine-tuning/>.
- [83] LangChain. *Llama2Chat LangChain*. python.langchain.com. URL: https://python.langchain.com/docs/integrations/chat/llama2_chat/.
- [84] LanguageTool. *How Many Words Should Be in a Sentence? — LanguageTool*. Blog, Jan. 2022. URL: <https://languagetool.org/insights/post/sentence-length/#:~:text=What%20Is%20the%20Ideal%20Length>.
- [85] eiTan LaVi. *Text Summarization Evaluation - BLEU vs ROUGE*. Stack Overflow, Aug. 2016. URL: <https://stackoverflow.com/questions/38045290/text-summarization-evaluation-bleu-vs-rouge>.
- [86] Ceshine Lee. *Use TextRank to Extract Most Important Sentences in Article*. Veritable, Apr. 2019. URL: <https://medium.com/the-artificial-impostor/use-textrank-to-extract-most-important-sentences-in-article-b8efc7e70b4>.
- [87] UK Legislation. *Legislation.gov.uk*. Legislation.gov.uk, 2022. URL: <https://www.legislation.gov.uk/>.
- [88] The Florida Legislature. *Statutes & Constitution : View Statutes : Online Sunshine*. www.leg.state.fl.us, 2023. URL: http://www.leg.state.fl.us/statutes/index.cfm?App_mode=Display_Statute&Search_String=&URL=0600-0699/0627/Sections/0627.4145.html.
- [89] Matthew Lelii. *Trust in government, UK - Office for National Statistics*. www.ons.gov.uk, July 2022. URL: [https://www.ons.gov.uk/peoplepopulationandcommunity/wellbeing/bulletins/trustinggovernmentuk/2022#:~:text=The%20majority%20\(58%25\)%20of](https://www.ons.gov.uk/peoplepopulationandcommunity/wellbeing/bulletins/trustinggovernmentuk/2022#:~:text=The%20majority%20(58%25)%20of).
- [90] Hanzhou Li et al. “Ethics of large language models in medicine and medical research”. In: *The Lancet Digital Health* (Apr. 2023). DOI: [10.1016/s2589-7500\(23\)00083-3](https://doi.org/10.1016/s2589-7500(23)00083-3).
- [91] Chin-Yew Lin. *ROUGE: A Package for Automatic Evaluation of Summaries*. July 2004. URL: <https://aclanthology.org/W04-1013.pdf>.
- [92] Google LLC. *rouge-score: Pure python implementation of ROUGE-1.5.5*. PyPI, July 2022. URL: <https://pypi.org/project/rouge-score/>.
- [93] Ahmed Lodhi. *How To Use ChatGPT To Summarize An Article: Quick & Easy Tips in 2023*. Niche Pursuits, Apr. 2023. URL: <https://www.nichepursuits.com/how-to-use-chatgpt-to-summarize-an-article/>.
- [94] Stephen M. Walker. *What is the ROUGE Score (Recall-Oriented Understudy for Gisting Evaluation)? — Klu*. klu.ai, July 2023. URL: <https://klu.ai/glossary/rouge-score>.
- [95] Pekka Malo et al. “Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts”. In: *Journal of the American Society for Information Science and Technology* (Apr. 2014). DOI: [10.1002/asi.23062](https://doi.org/10.1002/asi.23062).
- [96] MathWorks. *Explore Fairness Metrics for Credit Scoring Model - MATLAB & Simulink - MathWorks United Kingdom*. uk.mathworks.com. URL: <https://uk.mathworks.com/help/risk/explore-fairness-metrics-for-credit-scoring-model.html>.
- [97] Shilpi Mazumdar. *Exploring the Extractive Method of Text Summarization*. Analytics Vidhya, Mar. 2023. URL: <https://www.analyticsvidhya.com/blog/2023/03/exploring-the-extractive-method-of-text-summarization/>.
- [98] Glenda M. McClure. “Readability formulas: Useful or useless?” In: *IEEE Transactions on Professional Communication* PC-30.1 (1987), pp. 12–15. DOI: [10.1109/TPC.1987.6449109](https://doi.org/10.1109/TPC.1987.6449109).
- [99] Meta. *Llama 2 Family - a meta-llama Collection*. huggingface.co, Apr. 2024. URL: <https://huggingface.co/collections/meta-llama/llama-2-family-661da1f90a9d678b6f55773b>.
- [100] Microsoft. *Using ChatGPT to summarize text*. Microsoft 365, Apr. 2023. URL: <https://www.microsoft.com/en-us/microsoft-365-life-hacks/writing/using-chatgpt-summarizing-paraphrasing-text>.

- [101] Rada Mihalcea and Paul Tarau. *TextRank: Bringing Order into Texts*. 2004. URL: <https://web.eecs.umich.edu/~mihalcea/papers/mihalcea.emnlp04.pdf>.
- [102] Brent Mittelstadt, Sandra Wachter, and Chris Russell. “To protect science, we must use LLMs as zero-shot translators”. In: *Nature Human Behaviour* 7 (Nov. 2023), pp. 1830–1832. DOI: [10.1038/s41562-023-01744-0](https://doi.org/10.1038/s41562-023-01744-0). URL: <https://www.nature.com/articles/s41562-023-01744-0>.
- [103] Jack Morris. *language-tool-python: Checks grammar using LanguageTool*. PyPI, May 2020. URL: <https://pypi.org/project/language-tool-python/>.
- [104] Vishal Mysore. *Your First LLM Experiment*. www.linkedin.com, Dec. 2023. URL: <https://www.linkedin.com/pulse/your-first-llm-experiment-vishal-mysore-j55ec/>.
- [105] Kuldeep Nagi. *New Social Media and Impact of Fake News on Society*. Ssrn.com, 2018. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3258350.
- [106] Meera Nair. *Web scraping and topic modelling using K-means clustering*. Medium, Mar. 2024. URL: <https://medium.com/@miramnair/web-scraping-and-topic-modelling-using-k-means-clustering-9167d91461a0>.
- [107] Office for National Statistics. *Language, England and Wales - Office for National Statistics*. www.ons.gov.uk, 2022. URL: <https://www.ons.gov.uk/peoplepopulationandcommunity/culturalidentity/language/bulletins/languageenglandandwales/census2021>.
- [108] Ganymede Nil. *How to Segment Large Texts for Better Summarization*. Medium, Feb. 2023. URL: <https://medium.com/@ganymedenil/how-to-segment-large-texts-for-better-summarization-1e680a5eb980#:~:text=CDC%20splits%20the%20text%20based>.
- [109] Robert Noggle. “Manipulation in Politics”. In: *Oxford Research Encyclopedia of Politics* (Sept. 2021). DOI: [10.1093/acrefore/9780190228637.013.2012](https://doi.org/10.1093/acrefore/9780190228637.013.2012).
- [110] Ryan O’Connor. *Emergent Abilities of Large Language Models*. News, Tutorials, AI Research, Mar. 2023. URL: <https://www.assemblyai.com/blog/emergent-abilities-of-large-language-models/>.
- [111] OECD. *Transparency, communication and trust: The role of public communication in responding to the wave of disinformation about the new Coronavirus*. OECD, July 2020. URL: <https://www.oecd.org/coronavirus/policy-responses/transparency-communication-and-trust-the-role-of-public-communication-in-responding-to-the-wave-of-disinformation-about-the-new-coronavirus-bef7ad6e/>.
- [112] Ofcom. *Online Nation: interactive report*. Ofcom, May 2022. URL: <https://www.ofcom.org.uk/research-and-data/online-research/online-nation/interactive>.
- [113] Justin Olsson. *LLM-based summarization: A case study of human, Llama 2 70b and GPT-4 summarization quality*. Anyscale, Nov. 2023. URL: <https://www.anyscale.com/blog/llm-based-summarization-a-case-study-of-human-llama-2-70b-and-gpt-4-summarization-quality>.
- [114] OpenAI. *OpenAI API*. platform.openai.com. URL: <https://platform.openai.com/docs/models/overview>.
- [115] University of Oxford. *Legislation*. Oxford Law Faculty, Sept. 2015. URL: <https://www.law.ox.ac.uk/legal-research-and-mooting-skills-programme/legislation>.
- [116] University of Oxford. *Social media manipulation by political actors an industrial scale problem - Oxford report — University of Oxford*. www.ox.ac.uk, Jan. 2021. URL: <https://www.ox.ac.uk/news/2021-01-13-social-media-manipulation-political-actors-industrial-scale-problem-oxford-report>.
- [117] James Padolsey. *Using LLMs to parse and understand proposed legislation*. Medium, Aug. 2023. URL: <https://padolsey.medium.com/using-llms-to-parse-and-understand-proposed-legislation-9eec469d9830>.
- [118] Kishore Papineni et al. *BLEU: a Method for Automatic Evaluation of Machine Translation*. July 2002. URL: <https://aclanthology.org/P02-1040.pdf>.
- [119] UK Parliament. *Aviation (Offences) Act 2003*. Legislation.gov.uk, 2024. URL: <https://www.legislation.gov.uk/ukpga/2003/19/enacted?view=plain>.
- [120] UK Parliament. *Bills API*. bills-api.parliament.uk. URL: <https://bills-api.parliament.uk/index.html>.

- [121] UK Parliament. *Equality Act 2010*. legislation.gov.uk, 2010. URL: <https://www.legislation.gov.uk/ukpga/2010/15/contents>.
- [122] UK Parliament. *Female Genital Mutilation Act 2003*. Legislation.gov.uk, 2024. URL: <https://www.legislation.gov.uk/ukpga/2003/31/enacted?view=plain>.
- [123] UK Parliament. *Finance (No. 2) Act 2023*. Legislation.gov.uk, 2023. URL: <https://www.legislation.gov.uk/ukpga/2023/30/contents/enacted>.
- [124] UK Parliament. *Finance Act 2016*. Legislation.gov.uk, 2016. URL: <https://www.legislation.gov.uk/ukpga/2016/24/contents?view=plain>.
- [125] UK Parliament. *Glossary*. UK Parliament, 2019. URL: <https://www.parliament.uk/site-information/glossary/>.
- [126] UK Parliament. *Hansard*. Parliament.uk, 2019. URL: <https://hansard.parliament.uk/>.
- [127] UK Parliament. *How to petition against a private bill*. URL: <https://www.parliament.uk/about/how/laws/bills/private/privatebillpetition/>.
- [128] UK Parliament. *Knives Act 1997*. Legislation.gov.uk, 2024. URL: <https://www.legislation.gov.uk/ukpga/1997/21/enacted?view=plain>.
- [129] UK Parliament. *Public Order Act 2023*. Legislation.gov.uk, 2023. URL: <https://www.legislation.gov.uk/ukpga/2023/15/enacted>.
- [130] UK Parliament. *Regulating sex and sexuality: the 20th century*. UK Parliament, 2019. URL: <https://www.parliament.uk/about/living-heritage/transformingsociety/private-lives/relationships/overview/sexuality20thcentury/>.
- [131] UK Parliament. *What is a bill?* UK Parliament, 2019. URL: <https://www.parliament.uk/about/how/laws/bills/>.
- [132] Federico Pascual. *Getting Started with Sentiment Analysis using Python*. huggingface.co, Feb. 2022. URL: <https://huggingface.co/blog/sentiment-analysis-python>.
- [133] Mohammed Karimkhan Pathan. *Zero Shot, One Shot and Few Shot Learning - Prompt Engineering*. www.linkedin.com, Oct. 2023. URL: <https://www.linkedin.com/pulse/zero-shot-one-few-learning-prompt-engineering-pathan/>.
- [134] Pascoe Pleasence et al. *How People Understand and Interact with the Law*. 2015. URL: https://www.thelegaleducationfoundation.org/wp-content/uploads/2015/12/HPUIL_report.pdf.
- [135] Prodigal. *Extractive vs. Abstractive Summarization: How Does it Work?* www.prodigaltech.com. URL: <https://www.prodigaltech.com/blog/extractive-vs-abstractive-summarization-how-does-it-work>.
- [136] Darío Andrés Muñoz Prudent. *High quality very long text summarization with powerful 7B LLM's on high grade consumer GPU*. Medium, Mar. 2024. URL: <https://blog.cubed.run/introducing-efficient-and-simplified-long-text-summarization-with-powerful-7b-llms-and-high-grade-d56343bb2c5d>.
- [137] Jipeng Qiang et al. "LSBert: Lexical Simplification Based on BERT". In: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 29 (2021), pp. 3064–3076. DOI: [10.1109/TASLP.2021.3111589](https://doi.org/10.1109/TASLP.2021.3111589).
- [138] Sunil Ramlochan. *How Does Llama-2 Compare to GPT-4/3.5 and Other AI Language Models*. Prompt Engineering, Sept. 2023. URL: <https://promptengineering.org/how-does-llama-2-compare-to-gpt-and-other-ai-language-models/>.
- [139] Ujjwal Rani and Karambir Bidhan. "Comparative Assessment of Extractive Summarization: TextRank, TF-IDF and LDA". In: *Journal of scientific research* 65 (2021), pp. 304–311. DOI: [10.37398/jsr.2021.650140](https://doi.org/10.37398/jsr.2021.650140).
- [140] Keith Rayner and Susan A. Duffy. "Lexical complexity and fixation times in reading: Effects of word frequency, verb complexity, and lexical ambiguity". In: *Memory & Cognition* 14 (May 1986), pp. 191–201. DOI: [10.3758/bf03197692](https://doi.org/10.3758/bf03197692).
- [141] Readable. *Flesch Reading Ease and the Flesch Kincaid Grade Level*. Readable. URL: <https://readable.com/readability/flesch-reading-ease-flesch-kincaid-grade-level/#:~:text=The%20formula%20was%20developed%20in>.

- [142] Readable. *The Gunning Fog Index*. Readable. URL: <https://readable.com/readability/gunning-fog-index/>.
- [143] Readable. *The Raygor readability graph*. Readable, Mar. 2019. URL: <https://readable.com/blog/the-raygor-readability-graph/>.
- [144] Marta Recasens, Cristian Danescu-Niculescu-Mizil, and Dan Jurafsky. *Linguistic Models for Analyzing and Detecting Biased Language*. URL: <https://nlp.stanford.edu/pubs/neutrality.pdf>.
- [145] Janice Redish. “Readability formulas have even more limitations than Klare discusses”. In: *ACM Journal of Computer Documentation* 24 (Aug. 2000), pp. 132–137. DOI: [10.1145/344599.344637](https://doi.org/10.1145/344599.344637).
- [146] Nils Reimers. *Semantic Textual Similarity — Sentence-Transformers documentation*. www.sbert.net. URL: https://www.sbert.net/docs/usage/semantic_textual_similarity.html.
- [147] Deepak Reji. *d4data/bias-detection-model · Hugging Face*. huggingface.co, 2023. URL: <https://huggingface.co/d4data/bias-detection-model>.
- [148] Rewordify. *Rewordify.com Understand what you read*. Rewordify.com, 2019. URL: <https://rewordify.com/>.
- [149] Louise Bogéa Ribeiro et al. “Quantification of textual comprehension difficulty with an information theory-based algorithm”. In: *Palgrave Communications* 5 (Sept. 2019). DOI: [10.1057/s41599-019-0311-0](https://doi.org/10.1057/s41599-019-0311-0).
- [150] Leonard Richardson. *beautifulsoup4*. PyPI, Oct. 2019. URL: <https://pypi.org/project/beautifulsoup4/>.
- [151] Robert Henry Robins and David Crystal. *Language - Linguistic change*. Encyclopedia Britannica, Apr. 2024. URL: <https://www.britannica.com/topic/language/Linguistic-change>.
- [152] Mariana Romanyshyn. *Plainly speaking: a linguistic approach to simplifying complex words*. Plainly speaking: a linguistic approach to simplifying complex words — Grammarly Engineering Blog, Feb. 2020. URL: <https://www.grammarly.com/blog/engineering/plainly-speaking-a-linguistic-approach-to-simplifying-complex-words/#:~:text=Word%20frequency>.
- [153] Kei Sakaguchi et al. “WinoGrande: An Adversarial Winograd Schema Challenge at Scale”. In: *arXiv (Cornell University)* (July 2019). DOI: [10.48550/arxiv.1907.10641](https://doi.org/10.48550/arxiv.1907.10641).
- [154] “TF-IDF”. In: *Encyclopedia of Machine Learning*. Ed. by Claude Sammut and Geoffrey I. Webb. Boston, MA: Springer US, 2010, pp. 986–987. ISBN: 978-0-387-30164-8. DOI: [10.1007/978-0-387-30164-8_832](https://doi.org/10.1007/978-0-387-30164-8_832). URL: https://doi.org/10.1007/978-0-387-30164-8_832.
- [155] Phillip Schmid et al. *Llama 2 is here - get it on Hugging Face*. huggingface.co, July 2023. URL: <https://huggingface.co/blog/llama2>.
- [156] Roie Schwaber-Cohen. *Chunking Strategies for LLM Applications — Pinecone*. www.pinecone.io, June 2023. URL: <https://www.pinecone.io/learn/chunking-strategies/>.
- [157] Brian Scott. *The Gunning’s Fog Index (or FOG) Readability Formula*. ReadabilityFormulas.com, July 2023. URL: <https://readabilityformulas.com/the-gunnings-fog-index-or-fog-readability-formula/>.
- [158] James Scott. *What is the perfect sentence length?* Medium, Apr. 2019. URL: <https://medium.com/@scottydocs/what-is-the-perfect-sentence-length-4690ce8d5048>.
- [159] sebastianruder. *sebastianruder/NLP-progress*. GitHub, Sept. 2019. URL: <https://nlpprogress.com/english/simplification.html> (visited on 09/23/2019).
- [160] Ari Seff. *What are Transformer Neural Networks?* www.youtube.com, May 2021. URL: https://www.youtube.com/watch?v=XSSTuhyAmnI&ab_channel=AriSeff.
- [161] C. E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27 (Oct. 1948), pp. 623–656. DOI: [10.1002/j.1538-7305.1948.tb00917.x](https://doi.org/10.1002/j.1538-7305.1948.tb00917.x). URL: <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>.
- [162] Advait Siddharthan. “Syntactic Simplification and Text Cohesion”. In: *Research on Language and Computation* 4 (Mar. 2006), pp. 77–109. DOI: [10.1007/s11168-006-9011-1](https://doi.org/10.1007/s11168-006-9011-1).
- [163] Julien Simon. *Large Language Models: A New Moore’s Law?* huggingface.co, Oct. 2021. URL: <https://huggingface.co/blog/large-language-models>.
- [164] E Smith and Senter. *AUTOMATED READABILITY INDEX*. Nov. 1967. URL: <https://apps.dtic.mil/sti/tr/pdf/AD0667273.pdf> (visited on 04/06/2024).

- [165] Sheldon Smith. *Complex grammar*. www.eapfoundation.com, Apr. 2022. URL: <https://www.eapfoundation.com/writing/skills/grammar/#:~:text=It%20can%20be%20one%20of>.
- [166] The Law Society. *Public legal education*. www.lawsociety.org.uk, Mar. 2024. URL: <https://www.lawsociety.org.uk/campaigns/public-legal-education>.
- [167] Artifex Software. *PyMuPDF*. PyPI, May 2019. URL: <https://pypi.org/project/PyMuPDF/>.
- [168] George Spache. “A New Readability Formula for Primary-Grade Reading Materials”. In: *The Elementary School Journal* 53 (1953), pp. 410–413. URL: <https://www.jstor.org/stable/998915>.
- [169] Dayana Spagnuolo, Cesare Bartolini, and Gabriele Lenzini. *Metrics for Transparency*. Sept. 2016. URL: <https://core.ac.uk/download/pdf/78370936.pdf>.
- [170] Karen Steinhauser. *Everyone Is a Little Bit Biased*. www.americanbar.org, Mar. 2020. URL: https://www.americanbar.org/groups/business_law/resources/business-law-today/2020-april/everyone-is-a-little-bit-biased/.
- [171] Nisan Stiennon et al. “Learning to summarize from human feedback”. In: *arXiv:2009.01325 [cs]* (Feb. 2022). URL: <https://arxiv.org/abs/2009.01325>.
- [172] Aaron Swartz. *html2text: Turn HTML into equivalent Markdown-structured text*. PyPI, Jan. 2020. URL: <https://pypi.org/project/html2text/>.
- [173] Teerapaun Tanprasert and David Kauchak. *Flesch-Kincaid is Not a Text Simplification Evaluation Metric*. 2021. URL: <https://aclanthology.org/2021.gem-1.1.pdf>.
- [174] Rachel Tatman. *English Word Frequency*. www.kaggle.com, 2017. URL: <https://www.kaggle.com/datasets/ratman/english-word-frequency>.
- [175] Simon Taylor. *Are you aware of how literate your employees are?* www.ascento.co.uk, Jan. 2019. URL: <https://www.ascento.co.uk/blog/are-you-aware-of-how-literate-your-employees-are#:~:text=Readability%20data%20suggest%20that%20the>.
- [176] Chapman University. *Bias in AI*. www.chapman.edu, 2023. URL: <https://www.chapman.edu/ai/bias-in-ai.aspx>.
- [177] Princeton University. *WordNet — A Lexical Database for English*. Princeton.edu, 2019. URL: <https://wordnet.princeton.edu/>.
- [178] Kirti Vashee. *Understanding MT Quality: BLEU Scores*. www.rws.com, Apr. 2019. URL: <https://www.rws.com/blog/understanding-mt-quality-bleu-scores/#:~:text=But%20even%20two%20competent%20human>.
- [179] Ashish Vaswani et al. *Attention Is All You Need*. arXiv.org, June 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [180] Miguel de la Vega. *Understanding OpenAIs Temperature and Top p Parameters in Language Models*. Medium, Nov. 2023.
- [181] Sara Vincent. *Sentence length: why 25 words is our limit - Inside GOV.UK*. insidegovuk.blog.gov.uk, Aug. 2014. URL: <https://insidegovuk.blog.gov.uk/2014/08/04/sentence-length-why-25-words-is-our-limit/>.
- [182] Madeleine Watson. *The Prevalence of Dyslexia - Natalie Houalla*. Dyslexia Action, Oct. 2023. URL: <https://dyslexiaaction.org.uk/2023/10/the-prevalence-of-dyslexia/#:~:text=Currently%20C%20the%20British%20Dyslexia%20Association>.
- [183] Michael Webster and Jacob Schmitt. *LLM hallucinations: How to detect and prevent them with CI*. CircleCI, Jan. 2024. URL: <https://circleci.com/blog/llm-hallucinations-ci/#:~:text=An%20LLM%20hallucination%20occurs%20when>.
- [184] Jason Wei et al. *Emergent Abilities of Large Language Models*. Aug. 2022. URL: <https://openreview.net/pdf?id=yzkSU5zdwD#:~:text=2%20Emergent%20Abilities%20Definition&text=An%20ability%20is%20emergent%20if>.
- [185] Thomas Wood. *Large language models (LLM) and NLP: A new era of AI and ML has begun*. Fast Data Science, Apr. 2024. URL: [https://fastdatascience.com/llm-nlp/#:~:text=LLM%20and%20NLP%20\(natural%20language](https://fastdatascience.com/llm-nlp/#:~:text=LLM%20and%20NLP%20(natural%20language).
- [186] Wordsrated. *36 stats and facts on reading speed – WordsRated*. Wordsrated, Nov. 2021. URL: <https://wordsrated.com/reading-speed-statistics/>.

- [187] Wei Xu et al. “Optimizing Statistical Machine Translation for Text Simplification”. In: *Transactions of the Association for Computational Linguistics* 4 (Dec. 2016), pp. 401–415. DOI: [10.1162/tacL_a_00107](https://doi.org/10.1162/tacL_a_00107).
- [188] Yuhui Zhang, Ruocheng Wang, and Zhengping Zhou. *Improving Neural Abstractive Summarization via Reinforcement Learning with BERTScore*. URL: https://cs229.stanford.edu/proj2019aut/data/assignment_308832_raw/26632588.pdf.
- [189] Tianyi Zhang* et al. *BERTScore: Evaluating Text Generation with BERT*. openreview.net, Sept. 2019. URL: <https://openreview.net/forum?id=SkeHuCVFDr>.